

Kỹ thuật học tập tập thể trong bài toán dự báo chuỗi thời gian

Nguyễn Việt Tuấn Kiệt, Bùi Quang Phong,
Nguyễn Thái Hòa, Lưu Thịnh Khang, Nguyễn Thanh Tuyền

Chương trình tài năng - Khoa học máy tính K67
Trường Công nghệ Thông tin và Truyền Thông,
Đại học Bách khoa Hà Nội

Tóm tắt nội dung

Dữ liệu chuỗi thời gian ngày càng phổ biến, xuất hiện rộng rãi trong các lĩnh vực như kinh tế, tài chính, khí tượng, công nghiệp và y tế. Kéo theo đó, bài toán dự báo chuỗi thời gian trở thành một hướng nghiên cứu quan trọng với nhiều ứng dụng thực tiễn. Sự phát triển nhanh chóng của học máy và học sâu đã mang lại những mô hình có khả năng dự báo chính xác hơn. Tuy nhiên, do tính chất phi tuyến và phức tạp của dữ liệu, các mô hình đơn lẻ thường khó đạt hiệu quả tối ưu. Báo cáo này đề xuất học tập tập thể (ensemble learning) như một giải pháp tiềm năng nhằm cải thiện chất lượng dự báo bằng cách kết hợp sức mạnh của nhiều mô hình.

Mã nguồn dự án: [EL4TF: A Study on Ensemble Learning in Time-Series Forecasting](#)

1 Giới thiệu



Hình 1: Ví dụ chuỗi thời gian: giá mở cửa và khối lượng giao dịch của cổ phiếu theo ngày

Dữ liệu chuỗi thời gian xuất hiện phổ biến trong nhiều lĩnh vực như tài chính, kinh tế, khí tượng, y tế và công nghiệp, phản ánh sự biến đổi của hiện tượng theo thời gian. Việc dự báo chuỗi thời gian giúp đưa ra quyết định chính xác và kịp thời, đóng vai trò quan trọng trong thực tiễn. Tuy nhiên, do tính phi tuyến, nhiễu nhiễu và cấu trúc phụ thuộc phức tạp, việc xây dựng mô hình dự báo hiệu quả là một thách thức lớn.

Mặc dù các mô hình học máy và học sâu như ARIMA, TCN, LSTM, TFT đã đạt được nhiều tiến bộ, nhưng một mô hình đơn lẻ thường không đủ linh hoạt để thích ứng với mọi loại dữ liệu. Điều này thúc đẩy việc sử dụng học tập tập thể (ensemble learning) — kỹ thuật kết hợp nhiều mô hình nhằm tận dụng ưu điểm và bù trừ khuyết điểm của từng mô hình con. Các phương pháp phổ biến gồm Bagging, Boosting và Stacking.

Báo cáo này trình bày tổng quan về học tập tập thể trong bài toán dự báo chuỗi thời gian, giới thiệu các kỹ thuật chính, cách triển khai và minh họa hiệu quả thông qua thực nghiệm. Mục tiêu là làm rõ tiềm năng của ensemble learning trong việc cải thiện độ chính xác và tính ổn định của dự báo.

2 Cơ sở lý thuyết

2.1 Chuỗi thời gian

Định nghĩa 1. *Chuỗi thời gian* (time series) là một dãy các quan sát được thu thập liên tục hoặc theo từng khoảng thời gian xác định, trong đó mỗi quan sát được đánh dấu bởi mốc thời gian tương ứng.

Thông thường, ta ký hiệu chuỗi thời gian đã quan sát được dưới dạng

$$\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3, \dots, \mathbf{x}_t, \mathbf{x}_{t+1}, \dots$$

với $\mathbf{x}_t \in \mathbb{R}^d$ là giá trị hoặc vector giá trị tại thời điểm t và d là số chiều của dữ liệu hay số đặc trưng ghi nhận tại mỗi thời điểm.

Đặc điểm quan trọng nhất của chuỗi thời gian so với dữ liệu độc lập là mỗi phụ thuộc theo thứ tự thời gian: giá trị tại thời điểm t thường chịu ảnh hưởng trực tiếp (hoặc gián tiếp) từ các giá trị đã quan sát trước đó.

Trong dạng đơn biến (univariate), mỗi quan sát $\mathbf{x}_t$ chỉ là một số thực, ví dụ như nhiệt độ trung bình hàng ngày hoặc giá đóng cửa của một cổ phiếu. Dạng đa biến (multivariate), mỗi quan sát là một vector chứa nhiều thành phần, ví dụ như đồng thời ghi nhận nhiệt độ, độ ẩm và áp suất khí quyển.

Chuỗi thời gian còn có thể phân thành hai loại chính: chuỗi rời rạc, nơi các điểm dữ liệu cách đều nhau về mốc thời gian (ví dụ mỗi giờ, mỗi ngày), và chuỗi liên tục, nơi giá trị có thể được quan sát ở bất kỳ điểm nào trên trục thời gian, thường gặp trong các hệ thống điều khiển hoặc tín hiệu sinh học.

2.2 Tính chất Markov

Khi quan sát một chuỗi thời gian dưới dạng tập dữ liệu

$$\mathcal{D} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T\},$$

ta thực chất đang xem một đường mẫu (sample path) cụ thể của một quá trình ngẫu nhiên (stochastic process) $\{\mathbf{X}_t : t = 1, 2, \dots\}$. Trước khi thu thập dữ liệu, mỗi $\mathbf{X}_t$ là một biến ngẫu nhiên trên không gian xác suất $(\Omega, \mathcal{F}, P)$, trong đó

- Ω (space of outcomes) là tập hợp tất cả các kết quả có thể của thử nghiệm hoặc đường mẫu của quá trình.
- $\mathcal{F} \subseteq 2^\Omega$ (σ -algebra) là tập các sự kiện con của Ω sao cho nếu một sự kiện xảy ra, ta có thể gán xác suất cho nó; nó phải thoả mãn tính đóng bền với phần bù và hợp vô hạn dãy tập.
- $P : \mathcal{F} \rightarrow [0, 1]$ là phép đo xác suất gán xác suất cho mỗi sự kiện trong $\mathcal{F}$.

Khi thực hiện đo đạc, ta thu được giá trị cố định $\mathbf{X}_t = \mathbf{x}_t$, và toàn bộ chuỗi $(\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T)$ chỉ là một hiện thực cụ thể (realization) của quá trình đó. Việc coi chuỗi quan sát là một hiện thực của quá trình ngẫu nhiên cho phép chúng ta sử dụng các công cụ xác suất để ước lượng, kiểm định giả thuyết và gán khoảng tin cậy cho dự báo trong tương lai.

Để mô tả thông tin sẵn có đến từng thời điểm, lý thuyết quá trình ngẫu nhiên sử dụng khái niệm filtration. Một filtration là một họ các σ -đại số

$$\mathcal{F}_1 \subseteq \mathcal{F}_2 \subseteq \mathcal{F}_3 \subseteq \dots (\subseteq \mathcal{F}),$$

trong đó, mỗi $\mathcal{F}_t$ là một σ -đại số con của $\mathcal{F}$, chứa tất cả các sự kiện mà ta có thể xây dựng dựa trên giá trị của $\mathbf{X}_1, \mathbf{X}_2, \dots, \mathbf{X}_t$. Chẳng hạn, sự kiện “trung bình của $\mathbf{X}_1, \mathbf{X}_2, \mathbf{X}_3$ nhỏ hơn hoặc bằng c ” là một phần tử của $\mathcal{F}_3$, bởi vì nó chỉ phụ thuộc vào các giá trị đã biết tới thời điểm $t = 3$.

Nói cách khác, $\mathcal{F}_t$ biểu diễn “tất cả những gì ta biết” khi đã quan sát đến bước t . Khi nói “ước lượng dựa trên $\mathcal{F}_t$ ”, tức là mọi phép tính xác suất điều kiện chỉ được dùng thông tin trong $\mathcal{F}_t$ và không được phép truy cập vào dữ liệu tương lai.

Định nghĩa 2. Giữa vô vàn dạng phụ thuộc theo thời gian, *tính chất Markov* là giả thiết đơn giản nhưng rất hữu ích: một quá trình $\{\mathbf{X}_t : t = 1, 2, \dots\}$ được gọi là Markov bậc 1 nếu với mọi giá trị khả dĩ $\mathbf{x}$, ta có

$$P(\mathbf{X}_{t+1} = \mathbf{x} | \mathcal{F}_t) = P(\mathbf{X}_{t+1} = \mathbf{x} | \mathbf{X}_t = \mathbf{x}_t),$$

Điều này nghĩa là khi dự báo bước kế tiếp, toàn bộ thông tin cần thiết chỉ nằm trong trạng thái hiện tại $\mathbf{X}_t$, còn mọi thông tin về quá khứ xa hơn không bổ sung thêm hiểu biết nào.

Mở rộng khái niệm trên, thì Markov bậc p có nghĩa là

$$P(\mathbf{X}_{t+1} = \mathbf{x} | \mathcal{F}_t) = P(\mathbf{X}_{t+1} = \mathbf{x} | \mathbf{X}_t = \mathbf{x}_t, \mathbf{X}_{t-1} = \mathbf{x}_{t-1}, \dots, \mathbf{X}_{t-p+1} = \mathbf{x}_{t-p+1}),$$

tức là ta chỉ cần giữ lại p bước lịch sử thì đã có được toàn bộ thông tin để dự báo cho thời điểm kế tiếp.

Tính chất Markov bậc p trong dự báo chuỗi thời gian có ý nghĩa cốt lõi là mức “nhỏ” tối đa cần thiết của mô hình: thay vì phải biết toàn bộ lịch sử $\mathbf{X}_1, \mathbf{X}_2, \dots, \mathbf{X}_t$, ta chỉ cần giữ lại p bước gần nhất để dự báo cho $\mathbf{X}_{t+1}$. Giả thiết Markov bậc p khẳng định rằng mọi thông tin có ích cho việc dự báo tương lai đã được gom gọn trong p quan sát cuối cùng, cho phép ta vừa giữ đủ thông tin quan trọng để dự báo, vừa hạn chế kích thước mô hình sao cho việc xây dựng, ước lượng và áp dụng mô hình trở nên khả thi.

2.3 Dự báo chuỗi thời gian

a. Dự báo điểm

Giả sử ta đã thu thập được một tập dữ liệu gồm T quan sát về một chuỗi thời gian

$$\mathcal{D} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T\},$$

trường hợp đơn giản nhất của bài toán dự báo chuỗi thời gian là xác định giá trị tại H thời điểm tiếp theo, dựa trên giá trị đã quan sát tại p thời điểm liền trước đó, với giả thuyết là chuỗi thời gian này có tính chất Markov bậc p . Trong đó, p gọi là kích thước cửa sổ lịch sử (window size) và H là khoảng dự báo (forecast horizon).

Với mỗi thời điểm t , ta tạo thành một cặp dữ liệu:

$$(\mathbf{u}_t, \mathbf{y}_t) = \left((\mathbf{x}_t, \mathbf{x}_{t+1}, \dots, \mathbf{x}_{t+p-1}), (\mathbf{x}_{t+p}, \mathbf{x}_{t+p+1}, \dots, \mathbf{x}_{t+p+H-1}) \right)$$

Lúc này, nhiệm vụ dự báo tương đương với bài toán hồi quy điển hình của học có giám sát. Giả sử đã chọn được lớp các hàm số $\mathcal{F} = \{f_\theta : \theta \in \Theta\}$, nhiệm vụ là tìm

một ánh xạ $f_{\theta^*} : (\mathbb{R}^d)^p \rightarrow (\mathbb{R}^d)^H$ sao cho:

$$\theta^* = \arg \min_{\theta} \mathbb{E}_{(\mathbf{u}, \mathbf{y}) \sim \mathcal{P}} [L(\mathbf{y}, f_{\theta}(\mathbf{u}))]$$

với $L(\cdot, \cdot)$ là hàm mất mát, thể hiện sai khác giữa giá trị thực tế và giá trị dự báo bởi mô hình, chẳng hạn $L(\mathbf{y}, \hat{\mathbf{y}}) = \|\mathbf{y} - \hat{\mathbf{y}}\|_2^2$; và $\mathcal{P}$ là phân phối ẩn của dữ liệu.

Vì chỉ quan sát được $N = T - p - H + 1$ cặp dữ liệu $(\mathbf{u}_t, \mathbf{y}_t)$ nên nhiệm vụ có thể xem như tối thiểu hóa lỗi thực nghiệm

$$\theta^* = \arg \min_{\theta} \frac{1}{N} \sum_{t=1}^N L(\mathbf{y}_t, f_{\theta}(\mathbf{u}_t))$$

Kết quả là mô hình f_{θ^*} có thể được sử dụng. Trong trường hợp $H = 1$, bài toán xem như dự báo một bước (one-step). Giá trị tại thời điểm $T + 1$, nằm ngoài T quan sát mà ta đã biết, được dự báo bằng cách

$$\hat{\mathbf{x}}_{T+1} = f_{\theta^*}(\mathbf{x}_{T-p+1}, \mathbf{x}_{T-p}, \dots, \mathbf{x}_T) \quad (1)$$

Trong trường hợp $H > 1$, bài toán gọi là dự báo nhiều bước (multi-step). Giá trị tại các thời điểm $T + 1, T + 2, \dots, T + H$ được dự báo đồng thời bằng cách

$$(\hat{\mathbf{x}}_{T+1}, \hat{\mathbf{x}}_{T+2}, \dots, \hat{\mathbf{x}}_{T+H}) = f_{\theta^*}(\mathbf{x}_{T-p+1}, \mathbf{x}_{T-p+2}, \dots, \mathbf{x}_T) \quad (2)$$

Hoặc, áp dụng mô hình thu được theo phương pháp one-step ($H = 1$) để liên tục đưa ra các dự báo trong tương lai

$$\begin{aligned} \hat{\mathbf{x}}_{T+1} &= f_{\theta^*}(\mathbf{x}_{T-p+1}, \mathbf{x}_{T-p}, \dots, \mathbf{x}_{T-1}, \mathbf{x}_T) \\ \hat{\mathbf{x}}_{T+2} &= f_{\theta^*}(\mathbf{x}_{T-p+2}, \mathbf{x}_{T-p+1}, \dots, \mathbf{x}_T, \hat{\mathbf{x}}_{T+1}) \\ \hat{\mathbf{x}}_{T+3} &= f_{\theta^*}(\mathbf{x}_{T-p+3}, \mathbf{x}_{T-p+2}, \dots, \hat{\mathbf{x}}_{T+1}, \hat{\mathbf{x}}_{T+2}) \\ &\dots \end{aligned} \quad (3)$$

Rõ ràng, cách làm (2) không thể hiện rõ tính ràng buộc thời gian của $\hat{\mathbf{x}}_{T+2}$ đối với $\hat{\mathbf{x}}_{T+1}$, $\hat{\mathbf{x}}_{T+3}$ đối với $\hat{\mathbf{x}}_{T+2}$,... còn với (3), các giá trị dự báo sẽ ngày càng khác biệt so với giá trị thực tế bởi hiện tượng sai số tích lũy. Trong báo cáo này chủ yếu sử dụng phương pháp (1), và bài toán dự báo lúc này chỉ cần thực hiện cho một bước tiếp theo, nếu như đã biết giá trị chính xác của p bước trước đó.

Thực tế, đối với nhiệm vụ học, ta đã chuẩn bị sẵn một tập dữ liệu

$$\mathcal{D}_{\text{actual}} = \{\mathbf{x}_{T+1}, \mathbf{x}_{T+2}, \dots, \mathbf{x}_{T+K}\}$$

được dùng để kiểm tra tính hiệu quả của mô hình f_{θ^*} . Khi ấy, các giá trị dự báo được tính toán bởi

$$\begin{aligned} \hat{\mathbf{x}}_{T+1} &= f_{\theta^*}(\mathbf{x}_{T-p+1}, \mathbf{x}_{T-p}, \dots, \mathbf{x}_{T-1}, \mathbf{x}_T) \\ \hat{\mathbf{x}}_{T+2} &= f_{\theta^*}(\mathbf{x}_{T-p+2}, \mathbf{x}_{T-p+1}, \dots, \mathbf{x}_T, \mathbf{x}_{T+1}) \\ \hat{\mathbf{x}}_{T+3} &= f_{\theta^*}(\mathbf{x}_{T-p+3}, \mathbf{x}_{T-p+2}, \dots, \mathbf{x}_{T+1}, \mathbf{x}_{T+2}) \\ &\dots \\ \hat{\mathbf{x}}_{T+K} &= f_{\theta^*}(\mathbf{x}_{T-p+K}, \mathbf{x}_{T-p+K+1}, \dots, \mathbf{x}_{T+K-2}, \mathbf{x}_{T+K-1}) \end{aligned}$$

thành tập giá trị dự báo

$$\mathcal{D}_{\text{predict}} = \{\hat{\mathbf{x}}_{T+1}, \hat{\mathbf{x}}_{T+2}, \dots, \hat{\mathbf{x}}_{T+K}\}$$

Trong bài toán hồi quy (regression), ta thường đánh giá mô hình bằng các thước đo định lượng (metric) sai số giữa giá trị thực tế và giá trị dự đoán. Một số thước đo phổ biến như là:

(i) *Mean Absolute Error (MAE)*:

$$\text{MAE} = \frac{1}{K} \sum_{i=1}^K \frac{1}{d} \sum_{j=1}^d |\mathbf{x}_{T+i}^{(j)} - \hat{\mathbf{x}}_{T+i}^{(j)}|.$$

là trung bình sai số tuyệt đối trên tất cả các chiều và các mẫu, càng nhỏ càng tốt.

(ii) *Mean Squared Error (MSE)*:

$$\text{MSE} = \frac{1}{K} \sum_{i=1}^K \frac{1}{d} \sum_{j=1}^d \left(\mathbf{x}_{T+i}^{(j)} - \hat{\mathbf{x}}_{T+i}^{(j)} \right)^2.$$

là trung bình sai số bình phương toàn bộ các phần tử, càng nhỏ càng tốt.

(iii) *Root Mean Squared Error (RMSE)*:

$$\text{RMSE} = \sqrt{\text{MSE}} = \sqrt{\frac{1}{K} \sum_{i=1}^K \frac{1}{d} \sum_{j=1}^d \left(\mathbf{x}_{T+i}^{(j)} - \hat{\mathbf{x}}_{T+i}^{(j)} \right)^2}.$$

càng nhỏ càng tốt.

(iv) *Mean Absolute Percentage Error (MAPE)*:

$$\text{MAPE} = \frac{100\%}{K} \sum_{i=1}^K \frac{1}{d} \sum_{j=1}^d \left| \frac{\mathbf{x}_{T+i}^{(j)} - \hat{\mathbf{x}}_{T+i}^{(j)}}{\mathbf{x}_{T+i}^{(j)}} \right|.$$

càng nhỏ càng tốt.

(v) *R-squared (R^2)*:

$$R^2 = \frac{1}{d} \sum_{j=1}^d \left[1 - \frac{SS_{\text{res}}^{(j)}}{SS_{\text{tot}}^{(j)}} \right].$$

trong đó

$$SS_{\text{res}}^{(j)} = \frac{1}{K} \sum_{i=1}^K \left(\mathbf{x}_{T+i}^{(j)} - \hat{\mathbf{x}}_{T+i}^{(j)} \right)^2,$$

$$SS_{\text{tot}}^{(j)} = \frac{1}{K} \sum_{i=1}^K \left(\mathbf{x}_{T+i}^{(j)} - \bar{\mathbf{x}}_{T+i}^{(j)} \right)^2,$$

$$\bar{\mathbf{x}}_{T+i}^{(j)} = \frac{1}{K} \sum_{i=1}^K \mathbf{x}_{T+i}^{(j)}.$$

Dễ thấy $R^2 \in (-\infty, 1]$ và càng gần 1 thì càng tốt.

b. Dự báo phân phối

Một phát biểu tổng quát hơn cho bài toán dự báo chuỗi thời gian, khi mục tiêu không phải là trả về chỉ một hoặc các giá trị tương lai mà cần ước lượng toàn bộ phân phối. Giả sử $\{\mathbf{X}_t : t \in \mathbb{N}\}$ là một quá trình ngẫu nhiên trên không gian xác suất $(\Omega, \mathcal{F}, P)$ và $\mathcal{F}_t = \sigma(\mathbf{X}_1, \dots, \mathbf{X}_t)$ là filtration.

Tại thời điểm hiện tại T ta đã quan sát một đường mẫu

$$\mathcal{D} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T\} \subset \mathbb{R}^d.$$

Với kích thước cửa sổ lịch sử p và khoảng dự báo H , mục tiêu là ước lượng *phân phối có điều kiện*

$$P(\mathbf{X}_{T+1:T+H} \mid \mathbf{X}_{T-p+1:T} = \mathbf{x}_{T-p+1:T}),$$

trong đó $\mathbf{X}_{t_1:t_2} \stackrel{\text{def}}{=} (\mathbf{X}_{t_1}, \mathbf{X}_{t_1+1}, \dots, \mathbf{X}_{t_2})$ và $\mathbf{x}_{t_1:t_2}$ là giá trị đã quan sát.

Tương tự như (a), ta cũng xây dựng một tập huấn luyện

$$\mathcal{T} = \left\{ (\mathbf{u}_t, \mathbf{y}_t) = (\mathbf{x}_{t:t+p-1}, \mathbf{x}_{t+p:t+p+H-1}) : t = 1, \dots, T - p - H + 1 \right\}.$$

Khi đã chọn một họ phân phối $\mathcal{Q}_\Theta = \{Q_\theta : \theta \in \Theta\}$. Mục tiêu là tối ưu

$$\theta^* = \arg \min_{\theta \in \Theta} \frac{1}{|\mathcal{T}|} \sum_{(\mathbf{u}, \mathbf{y}) \in \mathcal{T}} S(Q_\theta(\cdot \mid \mathbf{u}), \mathbf{y}),$$

trong đó S là một *proper scoring rule*.

Khi đã có θ^* , ta có thể sử dụng mô hình theo nhiều cách: (i) lấy mẫu dữ liệu tại các thời điểm $\tilde{\mathbf{x}}_{T+1:T+H} \sim Q_{\theta^*}(\cdot \mid \mathbf{x}_{T-p+1:T})$, hoặc (ii) tính trực tiếp các đại lượng quan tâm như kỳ vọng $\mathbb{E}_{Q_{\theta^*}}[\cdot]$, phương sai $\text{Var}_{Q_{\theta^*}}[\cdot]$, phân vị α là $q_\alpha(\cdot)$.

3 Các mô hình tiêu biểu

3.1 Mô hình hồi quy

a. Mô hình tuyến tính tự hồi quy

Với giả thuyết rằng chuỗi thời gian có tính chất Markov bậc p , cùng với việc chọn $H = 1$ và $\mathcal{F} = \{f_\theta : \theta \in \Theta\}$ là lớp các mô hình tuyến tính. Với $f_\theta : (\mathbb{R}^d)^p \rightarrow \mathbb{R}^d$ được tham số hóa bởi $\theta = (\theta_0, \theta_1, \dots, \theta_p) \in \mathbb{R}^{p+1}$ ta mong muốn dự đoán

$$\begin{aligned} \mathbf{x}_{t+1} &\approx \hat{\mathbf{x}}_{t+1} = f_\theta(\mathbf{x}_{t-p+1}, \mathbf{x}_{t-p+2}, \dots, \mathbf{x}_t) \\ &= \theta_0 \mathbf{1}_d + \theta_1 \mathbf{x}_{t-p+1} + \theta_2 \mathbf{x}_{t-p+2} + \dots + \theta_p \mathbf{x}_t, \forall t \end{aligned} \quad (4)$$

trong đó $\mathbf{1}_d = [1, 1, \dots, 1] \in \mathbb{R}^d$.

Xét trên từng chiều của dữ liệu thì

$$\begin{aligned} \mathbf{x}_{t+1}^{(j)} &\approx \hat{\mathbf{x}}_{t+1}^{(j)} = f_\theta(\mathbf{x}_{t-p+1}^{(j)}, \mathbf{x}_{t-p+2}^{(j)}, \dots, \mathbf{x}_t^{(j)}) \\ &= \theta_0 + \theta_1 \mathbf{x}_{t-p+1}^{(j)} + \theta_2 \mathbf{x}_{t-p+2}^{(j)} + \dots + \theta_p \mathbf{x}_t^{(j)}, \forall j = \overline{1, d} \end{aligned} \quad (5)$$

điều này có nghĩa là, các chiều đều chia sẻ cùng một trọng số $\theta = (\theta_0, \theta_1, \dots, \theta_p)$. Nếu từng chiều của $\mathbf{x}_t$ là độc lập, không có quan hệ với nhau thì ta có thể giải quyết (5) bằng d bài toán hồi quy độc lập trên từng chiều. Tuy nhiên, trong thực tế, các chiều của $\mathbf{x}_t$ không hoàn toàn độc lập và chịu ảnh hưởng gián tiếp hoặc trực tiếp

của những đặc trưng trên các chiều khác. Ta cũng mong muốn học được mối quan hệ này một cách đồng thời trong quá trình tự hồi quy. Có thể viết lại (4) như sau

$$\begin{aligned}\mathbf{x}_{t+1} &\approx \hat{\mathbf{x}}_{t+1} = f_{\theta}(\mathbf{x}_{t-p+1}, \mathbf{x}_{t-p+2}, \dots, \mathbf{x}_t) \\ &= \theta_0 \mathbf{1}_d + \theta_1 \mathbf{I}_1 \mathbf{x}_{t-p+1} + \theta_2 \mathbf{I}_2 \mathbf{x}_{t-p+2} + \dots + \theta_p \mathbf{I}_p \mathbf{x}_t, \forall t\end{aligned}\quad (6)$$

với $\mathbf{I}_k \in \mathbb{R}^{d \times d}$ là ma trận đơn vị và $\mathbf{x}_t \in \mathbb{R}^{d \times 1}$. Đặt $\mathbf{W}_k = \theta_k \mathbf{I}_k \in \mathbb{R}^{d \times d}$ và $\mathbf{b} \in \mathbb{R}^d$ thì (6) thành

$$\begin{aligned}\mathbf{x}_{t+1} &\approx \hat{\mathbf{x}}_{t+1} = f_{\theta}(\mathbf{x}_{t-p+1}, \mathbf{x}_{t-p+2}, \dots, \mathbf{x}_t) \\ &= \mathbf{b} + \mathbf{W}_1 \mathbf{x}_{t-p+1} + \mathbf{W}_2 \mathbf{x}_{t-p+2} + \dots + \mathbf{W}_p \mathbf{x}_t, \forall t\end{aligned}\quad (7)$$

Như vậy, (7) là một biểu diễn tổng quát hơn và các $\mathbf{W}_k$ không nhất thiết cần chọn đặc biệt là $\theta_k \mathbf{I}_k$ như trước, có thể là bất kỳ ma trận $d \times d$ nào, $\mathbf{b}$ cũng là bất kỳ vector d chiều nào. Ta có thể ký hiệu

$$\begin{aligned}\mathbf{u}_t &= [\mathbf{x}_{t-p+1}^{(1)}, \dots, \mathbf{x}_{t-p+1}^{(d)}, \mathbf{x}_{t-p+2}^{(1)}, \dots, \mathbf{x}_{t-p+2}^{(d)}, \dots, \mathbf{x}_t^{(1)}, \dots, \mathbf{x}_t^{(d)}] \\ &= \text{flatten}[\mathbf{x}_{t-p+1}, \mathbf{x}_{t-p+2}, \dots, \mathbf{x}_t] \in \mathbb{R}^{1 \times (dp)}, \forall t\end{aligned}$$

là một vector trải phẳng của tất cả đặc trưng từ toàn bộ của số lịch sử, và

$$\mathbf{W} = [\mathbf{W}_1 \mid \mathbf{W}_2 \mid \dots \mid \mathbf{W}_p]^{\top} \in \mathbb{R}^{(dp) \times d}$$

là một ma trận khối tạo thành bằng việc ghép tất cả các ma trận trọng số theo chiều dọc. Toàn bộ mô hình trở thành

$$\mathbf{x}_{t+1} \approx \hat{\mathbf{x}}_{t+1} = f_{\mathbf{b}, \mathbf{W}}(\mathbf{u}_t) = \mathbf{b} + \mathbf{u}_t \mathbf{W}$$

Có thể ký hiệu $\tilde{\mathbf{u}}_t = [1, \mathbf{u}_t] \in \mathbb{R}^{1 \times (dp+1)}$, $\tilde{\mathbf{W}} = [\mathbf{b} \mid \mathbf{W}]^{\top} \in \mathbb{R}^{(dp+1) \times d}$ và

$$f_{\tilde{\mathbf{W}}}(\tilde{\mathbf{u}}_t) = \tilde{\mathbf{u}}_t \tilde{\mathbf{W}}, \forall t = p, T-1$$

(i) Phương pháp hồi quy OLS

Mục tiêu là tối thiểu hóa hàm lỗi trên tập huấn luyện có $N = T - p$ cặp dữ liệu

$$\tilde{\mathbf{W}}^* = \arg \min_{\tilde{\mathbf{W}}} \frac{1}{N} \sum_{t=p}^{T-1} \|\tilde{\mathbf{u}}_t \tilde{\mathbf{W}} - \mathbf{x}_{t+1}\|_2^2$$

Khi viết chồng các cột theo cách

$$\mathbf{U} = \begin{bmatrix} \tilde{\mathbf{u}}_p \\ \tilde{\mathbf{u}}_{p+1} \\ \vdots \\ \tilde{\mathbf{u}}_{T-1} \end{bmatrix} \in \mathbb{R}^{N \times (dp+1)}, \quad \mathbf{Y} = \begin{bmatrix} \mathbf{x}_{p+1} \\ \mathbf{x}_{p+2} \\ \vdots \\ \mathbf{x}_T \end{bmatrix} \in \mathbb{R}^{N \times d}.$$

Bài toán trở thành

$$\tilde{\mathbf{W}}^* = \arg \min_{\tilde{\mathbf{W}}} \frac{1}{N} \|\mathbf{U} \tilde{\mathbf{W}} - \mathbf{Y}\|_F^2, \quad (8)$$

với $\|\cdot\|_F$ là chuẩn Frobenius.

Để cho dễ dàng ghi nhớ, và phù hợp với cài đặt của thư viện `sklearn` (`scikit-learn`) thì: $\mathbf{U}$ có kích thước (`n_samples`, `n_features`), $\tilde{\mathbf{W}}$ có kích thước (`n_features + 1`, `n_tasks`) bao gồm bias $\mathbf{b}$ có kích thước (`n_tasks`,) cùng với $\mathbf{W}$ có kích thước (`n_features`, `n_tasks`) và $\mathbf{Y}$ có kích thước (`n_samples`, `n_tasks`). Trong đó

- `n_samples` là số lượng mẫu dữ liệu huấn luyện, ký hiệu là N_s ,
- `n_features` là số lượng đặc trưng, ký hiệu là N_f ,
- `n_tasks/n_outputs/n_targets` là số chiều của đầu ra, ký hiệu là N_t .

(ii) Phương pháp hồi quy Ridge

Phương pháp như (8) có thể mở rộng bằng việc thêm một đại lượng hiệu chỉnh để trở thành *bài toán hồi quy Ridge*

$$\tilde{\mathbf{W}}^* = \arg \min_{\tilde{\mathbf{W}}} \left[\frac{1}{N_t} \|\mathbf{U}\tilde{\mathbf{W}} - \mathbf{Y}\|_F^2 + \lambda \|\mathbf{W}\|_F^2 \right]$$

theo thông lệ, không phạt cột $\mathbf{b}$ trong triển khai.

(iii) Phương pháp hồi quy Lasso

Khi số lượng đặc trưng tăng lên, một số đặc trưng sẽ trở nên không thực sự quan trọng, *hồi quy Lasso* làm thưa ma trận trọng số để giữ lại những đặc trưng ảnh hưởng nhất. Trong thực tế, có 2 kiểu phạt được triển khai, bao gồm: *Lasso độc lập đầu ra* sử dụng chuẩn L_1 truyền thống, áp dụng riêng lẻ cho từng hệ số

$$\tilde{\mathbf{W}}^* = \arg \min_{\tilde{\mathbf{W}}} \left[\frac{1}{N_s} \|\mathbf{U}\tilde{\mathbf{W}} - \mathbf{Y}\|_F^2 + \lambda \sum_{i=1}^{N_f} \sum_{j=1}^{N_t} |\mathbf{W}_{i,j}| \right]$$

tương đương N_t bài toán Lasso đơn biến độc lập; và *multi-task Lasso* sử dụng chuẩn $L_{2,1}$, tức là chuẩn L_2 trong task và L_1 qua các feature

$$\tilde{\mathbf{W}}^* = \arg \min_{\tilde{\mathbf{W}}} \left[\frac{1}{N_s} \|\mathbf{U}\tilde{\mathbf{W}} - \mathbf{Y}\|_F^2 + \lambda \sum_{i=1}^{N_f} \sqrt{\sum_{j=1}^{N_t} \mathbf{W}_{i,j}^2} \right]$$

tức là toàn bộ một cột (một đặc trưng) bị loại bỏ đồng thời cho mọi biến đầu ra.

(iv) Phương pháp hồi quy ElasticNet

Một phiên bản phối hợp hiệu quả của Ridge và Lasso là *ElasticNet*, tương tự như Lasso, cũng có 2 triển khai bao gồm: *ElasticNet độc lập đầu ra* như sau

$$\tilde{\mathbf{W}}^* = \arg \min_{\tilde{\mathbf{W}}} \left[\frac{1}{N_s} \|\mathbf{U}\tilde{\mathbf{W}} - \mathbf{Y}\|_F^2 + \lambda \left((1 - \rho) \sum_{i=1}^{N_f} \sum_{j=1}^{N_t} |\mathbf{A}_{i,j}| + \rho \|\mathbf{A}\|_F^2 \right) \right]$$

trong đó, $\rho = 1$ thì bài toán tương đương với hồi quy Ridge và $\rho = 0$ thì trở thành hồi quy Lasso; *multi-task ElasticNet* phát biểu dưới dạng

$$\tilde{\mathbf{W}}^* = \arg \min_{\tilde{\mathbf{W}}} \left[\frac{1}{N_s} \|\mathbf{U}\tilde{\mathbf{W}} - \mathbf{Y}\|_F^2 + \lambda \left((1 - \rho) \sum_{i=1}^{N_f} \sqrt{\sum_{j=1}^{N_t} \mathbf{W}_{i,j}^2} + \rho \|\mathbf{W}\|_F^2 \right) \right]$$

Ngoài việc sử dụng các mô hình mạnh mẽ thì việc tăng tiến số lượng đặc trưng cũng có thể cải thiện được hiệu quả dự báo. Hiện tại, một đặc trưng $\mathbf{u}_t$ chỉ đơn thuần xây dựng dựa trên p giá trị lịch sử được trải phẳng, mỗi vector có d chiều, tổng cộng tạo thành vector $\mathbf{u}_t$ có dp chiều. Một cách tổng quát hơn, ta có thể sử dụng thêm nhiều thông tin phi tuyến có thể thu được từ $[\mathbf{x}_{t-p+1}, \mathbf{x}_{t-p+2}, \dots, \mathbf{x}_t]$ rồi nối nó vào vector $\mathbf{u}_t$, chẳng hạn: phương sai, trung bình, skewness, kurtosis,... miễn là giá trị

đó có thể tính được dựa trên dãy $[\mathbf{x}_{t-p+1}, \mathbf{x}_{t-p+2}, \dots, \mathbf{x}_t]$ để không mâu thuẫn với giả thuyết Markov bậc p mà ta đã đặt ra. Một khi số đặc trưng tăng lên, cũng đòi hỏi kỹ thuật lựa chọn đặc trưng tinh tế để tránh hiện tượng quá khớp (overfitting).

Một số mô hình đặc biệt như ARIMA vốn nổi tiếng trong nhiệm vụ dự báo chuỗi thời gian. Tuy nhiên báo cáo này chủ yếu tiếp cận theo góc nhìn học máy, tức lớp các mô hình có thể được gọi từ thư viện `sklearn` hoặc xây dựng được dựa trên PyTorch.

b. Mô hình cây quyết định

(i) Cây quyết định

Cây quyết định nhị phân (Decision Tree Regressor) xây dựng một cấu trúc cây, tại đó mỗi nút trong (internal node) lưu một điều kiện tách (split), và mỗi nút lá (leaf node) lưu giá trị dự đoán cho các mẫu rơi vào lá đó.

Đã có nhiều thuật toán nổi tiếng được sử dụng để xây dựng cây quyết định như ID3, C4.5, CART, CHAID với những ưu và nhược điểm riêng. Tuy nhiên báo cáo này sử dụng thuật toán CART (Classification And Regression Trees) được sử dụng trong thư viện `sklearn`. Mục tiêu của thuật toán là tìm cách chia dữ liệu (các split) sao cho trung bình sai số bình phương (MSE) nhỏ nhất.

Tập dữ liệu huấn luyện được xem xét có dạng

$$\mathcal{T} = \left\{ (\mathbf{u}_i, \mathbf{y}_i) : i = \overline{1, N_s} \right\}$$

trong $\mathbf{u}_i \in \mathbb{R}^{N_f}$ là vector đặc trưng đã trải phẳng, hơn nữa có thể bao gồm các đặc trưng khác còn $\mathbf{y}_i \in \mathbb{R}^{N_t}$ là vector cần dự đoán.

Ở mỗi nút, với tập mẫu $\mathcal{S} \subset \mathcal{T}$, thuật toán thử mọi đặc trưng $j \in \{1, 2, \dots, N_f\}$ và ngưỡng tách $s \in \mathbb{R}$ để phân chia $\mathcal{S}$ thành 2 tập con

$$\begin{aligned} \mathcal{S}_L(j, s) &= \left\{ (\mathbf{u}_i, \mathbf{y}_i) \in \mathcal{S} : (\mathbf{u}_i)_j < s \right\} \\ \mathcal{S}_R(j, s) &= \left\{ (\mathbf{u}_i, \mathbf{y}_i) \in \mathcal{S} : (\mathbf{u}_i)_j \geq s \right\} \end{aligned}$$

Với mỗi phép chia (j, s) , tính tổng sai số (SSE) trên nhãn

$$\text{SSE}(j, s) = \sum_{(\mathbf{u}, \mathbf{y}) \in \mathcal{S}_L} \|\mathbf{y} - \bar{\mathbf{y}}_L\|_2^2 + \sum_{(\mathbf{u}, \mathbf{y}) \in \mathcal{S}_R} \|\mathbf{y} - \bar{\mathbf{y}}_R\|_2^2$$

trong đó

$$\bar{\mathbf{y}}_L = \frac{1}{|\mathcal{S}_L|} \sum_{(\mathbf{u}, \mathbf{y}) \in \mathcal{S}_L} \mathbf{y}, \quad \bar{\mathbf{y}}_R = \frac{1}{|\mathcal{S}_R|} \sum_{(\mathbf{u}, \mathbf{y}) \in \mathcal{S}_R} \mathbf{y}$$

Thuật toán CART chọn cặp (j^*, s^*) sao cho

$$(j^*, s^*) = \arg \min_{(j, s)} \text{SSE}(j, s)$$

Quá trình này lặp lại đệ quy cho hai tập con $\mathcal{S}_L(j^*, s^*)$ và $\mathcal{S}_R(j^*, s^*)$, đến khi thỏa điều kiện dừng.

Khi thực hiện dự đoán với mẫu mới, cây quyết định dựa trên điều kiện tại mỗi lá và đưa mẫu đó về với lá mà nó thỏa mãn và gán nhãn (ở đây là giá trị dự đoán) cho mẫu này bằng nhãn đã gán với lá đó. Đây chính một trong những hạn chế cốt lõi của DT cho hồi quy: mỗi lá chỉ chứa một giá trị cố định, thường là trung bình nhãn của các mẫu huấn luyện rơi vào lá đó. Vì chỉ học trung bình trong từng vùng, cây không thể dự đoán giá trị ngoài phạm vi nhãn đã thấy – mọi dự đoán luôn nằm trong khoảng min–max của nhãn huấn luyện, cây DT do đó không thể ngoại suy (extrapolation).

Mô hình cây quyết định như vậy không thực sự phù hợp cho bài toán dự báo chuỗi thời gian vì bản chất thay đổi thất thường của dữ liệu theo thời gian, hoàn toàn có thể vượt ra khỏi giới hạn của nó trong lịch sử quan sát.

c. Mô hình mạng liên kết

Mạng RVFL (Random Vector Functional Link Network) là một loại mạng truyền thẳng (feedforward neural network) sử dụng các đặc trưng ẩn ngẫu nhiên, không cần lan truyền ngược để huấn luyện và nhanh chóng, đơn giản trong triển khai.

Trong phần này, ta vẫn sử dụng ký hiệu như đã định nghĩa trong (a) với tập huấn luyện $\mathcal{D} = (\mathbf{U}, \mathbf{Y})$ bao gồm: $\mathbf{U} \in \mathbb{R}^{N_s \times N_f}$ và $\mathbf{Y} \in \mathbb{R}^{N_s \times N_t}$. Trong đó N_s là số mẫu, N_f là số đặc trưng và N_t là số chiều đầu ra mong muốn.

(i) Kiến trúc RVFL

Kiến trúc *RVFL cơ bản* sử dụng một lớp ẩn duy nhất với m nút, hàm kích hoạt φ (chẳng hạn ReLU, sigmoid). Trước hết, các trọng số khởi tạo ngẫu nhiên

$$\mathbf{W} \in \mathbb{R}^{N_f \times m}, \quad \mathbf{b} \in \mathbb{R}^{1 \times m}$$

tuân theo phân phối đều hoặc phân phối chuẩn, và được giữ cố định.

Đầu ra tầng ẩn (đặc trưng ẩn, trừu tượng) được tính toán bằng cách

$$\mathbf{H} = \varphi(\mathbf{UW} + \mathbf{1}_{N_s} \mathbf{b}) \in \mathbb{R}^{N_s \times m}$$

trong đó $\mathbf{1}_{N_s} = [1, 1, \dots, 1]^\top \in \mathbb{R}^{N_s \times 1}$, sau đó được nối với ma trận đầu vào

$$\mathbf{Z} = \begin{bmatrix} \mathbf{U} & \mathbf{H} \end{bmatrix} \in \mathbb{R}^{N_s \times (N_f + m)}$$

tương đương số đặc trưng đã tăng thêm m với hi vọng m đặc trưng này thể hiện được một tính chất ẩn nào đó của dữ liệu thông qua trọng số ngẫu nhiên.

Cuối cùng, bài toán học tương đương hồi quy Ridge, với $\mathbf{B} \in \mathbb{R}^{(N_f + m) \times N_t}$ cần tìm

$$\mathbf{B}^* = \arg \min_{\mathbf{B}} \left\| \mathbf{ZB} - \mathbf{Y} \right\|_F^2 + \lambda \left\| \mathbf{B} \right\|_F^2,$$

có nghiệm cụ thể $\mathbf{B}^* = (\mathbf{Z}^\top \mathbf{Z} + \lambda \mathbf{I})^{-1} \mathbf{Z}^\top \mathbf{Y}$.

Vector dự báo cho một đầu vào mới $\mathbf{u} \in \mathbb{R}^{1 \times N_f}$ tính bởi

$$\hat{\mathbf{y}} = \begin{bmatrix} \mathbf{u} & \varphi(\mathbf{uW} + \mathbf{b}) \end{bmatrix} \mathbf{B}^* \in \mathbb{R}^{1 \times N_t}$$

(ii) Kiến trúc dRVFL

Biến thể đầu tiên của mạng RVFL là *dRVFL* (*Deep RVFL*), với việc tăng thêm số lượng lớp ẩn. Giả sử có L lớp ẩn, tầng ẩn tiếp theo trong mạng nhận đầu vào là đầu ra của tầng ẩn trước đó

$$\mathbf{H}^{(0)} = \mathbf{U}$$

$$\mathbf{H}^{(\ell)} = \varphi\left(\mathbf{H}^{(\ell-1)}\mathbf{W}^{(\ell)} + \mathbf{1}_{N_s}\mathbf{b}^{(\ell)}\right) \in \mathbb{R}^{N_s \times m_\ell}$$

với $\mathbf{W}^{(\ell)} \in \mathbb{R}^{(d+\sum_{k<\ell} m_k) \times m_\ell}$ được tạo ngẫu nhiên. Mỗi $\mathbf{H}^{(\ell)}$ được kỳ vọng biểu diễn một thông tin sâu sắc về dữ liệu, tại các tầng càng sâu thì mức độ trừu tượng càng mạnh mẽ.

Đầu ra cuối cùng tổng hợp cả L lớp ẩn

$$\mathbf{Z} = \left[\mathbf{U} \mid \mathbf{H}^{(1)} \mid \dots \mid \mathbf{H}^{(L)} \right] \in \mathbb{R}^{N_s \times (N_f + \sum_{\ell=1}^L m_\ell)}$$

tương đương việc ta đã bổ sung thêm $\sum_{\ell=1}^L m_\ell$ đặc trưng ẩn.

Bài toán học vẫn sử dụng hồi quy Ridge với nghiệm đã đề cập trước đó.

(iii) Kiến trúc edRVFL

Biến thể mạnh mẽ nhất của RVFL cho đến nay là *edRVFL* (*Ensemble Deep RVFL*) bằng việc kết hợp ý tưởng ensemble (tập hợp nhiều mô hình) với dRVFL để gia tăng tính ổn định và độ chính xác.

Ý tưởng ở đây rất đơn giản. Trước hết khởi tạo M mô hình dRVFL độc lập, mỗi mô hình m có: kiến trúc (số tầng, số nút mỗi tầng) giống hoặc khác nhau; các ma trận trọng số $\mathbf{W}^{(\ell,m)}$, $\mathbf{b}^{(\ell,m)}$ lấy ngẫu nhiên một cách độc lập. Tiếp theo, huấn luyện từng mô hình m thu được trọng số $\mathbf{B}^{*(m)}$.

Với một đầu vào mới $\mathbf{u}$, dự báo cuối cùng tạo thành bằng việc lấy tổ hợp tuyến tính của toàn bộ m dự báo từ các mô hình.

$$\hat{\mathbf{y}} = \sum_{m=1}^M \alpha_m \mathbf{Z}^{(m)} \mathbf{B}^{*(m)}, \quad \sum_{m=1}^M \alpha_m = 1$$

d. Mô hình mạng tích chập

Mạng tích chập (Convolutional Neural Network – CNN) là một kiến trúc mạng sâu được thiết kế để khai thác các đặc trưng cục bộ trong dữ liệu thông qua phép toán tích chập. CNN đã chứng minh hiệu quả vượt trội trong các bài toán thị giác máy tính, đặc biệt là xử lý ảnh – nơi mà mối liên kết không gian giữa các điểm ảnh là thông tin then chốt.

Trong báo cáo này, CNN được mở rộng áp dụng cho dữ liệu chuỗi thời gian. Quan điểm ở đây cho rằng ảnh là biểu diễn hai chiều của tín hiệu ánh sáng, còn chuỗi thời gian có thể được xem như “ảnh một chiều” – ghi lại sự biến thiên của tín hiệu theo thời gian. Nhờ khả năng học các mẫu cục bộ và bất biến tịnh tiến, CNN hứa hẹn là công cụ mạnh mẽ cho việc mô hình hóa chuỗi thời gian. Phiên bản CNN cho dự báo chuỗi thời gian được biết đến với tên gọi *mạng tích chập thời gian* (Temporal Convolutional Network – TCN), để phân biệt với mạng CNN vốn được dùng cho ảnh. Sự khác nhau cơ bản ở đây là, CNN cho xử lý ảnh sử dụng phép tích chập 2D, trong khi CNN cho xử lý chuỗi thời gian sử dụng phép tích chập 1D.

(i) Kiến trúc TCN

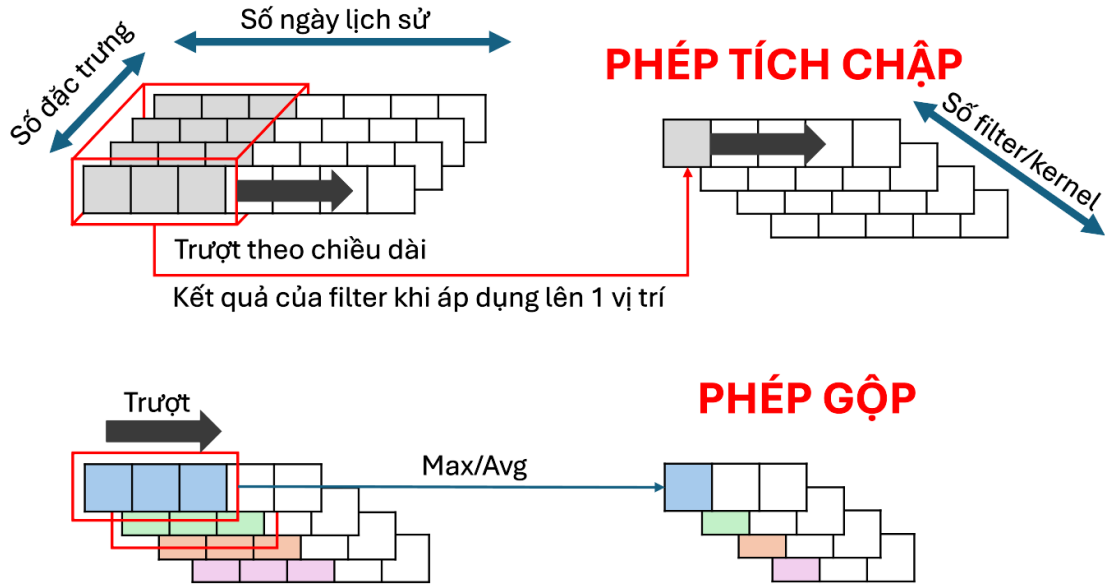
Với các ký hiệu đã sử dụng: $\mathbf{U} \in \mathbb{R}^{N_s \times N_f}$ và $\mathbf{Y} \in \mathbb{R}^{N_s \times N_t}$, trong đó $N_f = d \times p$ đến từ việc trải phẳng p vector, mỗi vector có d chiều. Ta sẽ tái định hình (reshape) $\mathbf{U}$ từ một ma trận 2 chiều thành một tensor 3 chiều

$$\mathbf{X}^{(0)} \in \mathbb{R}^{N_s \times C_0 \times L_0},$$

với $C_0 = d$ gọi là số kênh đầu vào và $L_0 = p$ là độ dài chuỗi nơi phép toán tích chập sẽ trượt qua. Triển khai trong PyTorch tương đương đầu vào là tensor với kích thước (batch_size, n_dimensions, window_size) với batch_size là số mẫu dữ liệu lan truyền tiến qua mạng cùng lúc trong một lần xử lý, n_dimensions là số chiều của một mẫu dữ liệu, window_size là kích thước số ngày lịch sử được sử dụng cho việc dự báo.

Mạng CNN thông thường được cấu tạo với tổng cộng B khối (block). Mỗi khối b trong mạng cho ra $\mathbf{X}^{(b)} \in \mathbb{R}^{N_s \times C_b \times L_b}$ cũng là một tensor 3 chiều, mà mỗi tensor nằm dọc theo chiều kênh gọi là bản đồ đặc trưng (feature map), tổng cộng có C_b kênh và mỗi kênh dài L_b .

Mỗi khối được cấu tạo từ các lớp (layer), mỗi lớp đại diện cho một toán tử trích xuất đặc trưng từ dữ liệu. Trong triển khai đơn giản nhất, một khối thường có thứ tự các lớp với mở đầu là phép tích chập, theo sau bởi hàm kích hoạt, tiếp theo có thể là chuẩn hóa theo lô (batch normalization) và ngắt bỏ (dropout) tuy nhiên không bắt buộc, cuối cùng kết thúc với phép gộp (pooling).



Hình 2: Kiến trúc TCN

Lớp tích chập của khối thứ b nhận $\mathbf{X}^{(b-1)} \in \mathbb{R}^{N_s \times C_{b-1} \times L_{b-1}}$ làm đầu vào, sử dụng C_b bộ lọc (filter/kernel), mỗi bộ lọc có chiều dài K_b . Tập hợp lại thành một tensor trọng số

$$\mathbf{W}^{(b)} \in \mathbb{R}^{C_b \times C_{b-1} \times K_b}, \quad \mathbf{b}^{(b)} \in \mathbb{R}^{C_b},$$

Mẫu thứ i , khi áp dụng bộ lọc f và vị trí thời gian j trên chuỗi được tính bằng

$$\mathbf{Conv}_{i,f,j}^{(b)} = \sum_{c=1}^{C_{b-1}} \sum_{k=0}^{K_b-1} \mathbf{W}_{f,c,k}^{(b)} \mathbf{X}_{i,c,j+k}^{(b-1)} + \mathbf{b}_f^{(b)},$$

với $L_b^{\text{conv}} = L_{b-1} - (K_b - 1)$ dễ dàng nhận thấy và σ là hàm kích hoạt.

Phép tích chập tổng quát sử dụng thêm 3 tham số: padding P (đệm) là việc chèn P giá trị 0 vào 2 đầu chuỗi trước khi tích chập nhằm kiểm soát độ dài đầu ra; stride S (bước nhảy) kiểm soát số vị trí mà filter di chuyển giữa hai phép toán tích chập liên tiếp giúp giảm độ dài feature map và làm thưa quan sát; dilation D giãn khoảng cách giữa các phần tử của bộ lọc, cho phép mở rộng vùng quan sát mà không tăng kích thước bộ lọc.

Khi có 3 tham số (P_b, S_b, D_b) , công thức chung trở thành:

$$\mathbf{Conv}_{i,f,j}^{(b)} = \sum_{c=1}^{C_{b-1}} \sum_{k=0}^{K_b-1} \mathbf{W}_{f,c,k}^{(b)} \mathbf{X}_{i,c,jS_b+kD_b-P_b}^{(b-1)} + \mathbf{b}_f^{(b)},$$

thu được độ dài đầu ra là

$$L_b^{\text{conv}} = \left\lfloor \frac{L_{b-1} + 2P_b - D_b(K_b - 1) - 1}{S_b} \right\rfloor + 1$$

Kết thúc lớp tích chập, đặc trưng thu được là $\mathbf{Conv}^{(b)} \in \mathbb{R}^{N_s \times C_b \times L_b^{\text{conv}}}$, vẫn là một tensor 3 chiều với lưu ý rằng, một bộ lọc chỉ tạo ra 1 bản đồ đặc trưng (feature map) và việc áp dụng C_b bộ lọc tạo ra tổng cộng C_b bản đồ đặc trưng nằm dọc theo chiều kênh (channel).

Theo sau lớp tích chập, hàm kích hoạt σ được sử dụng, chẳng hạn: tanh, ReLU, ... tạo ra đặc trưng cùng kích thước

$$\mathbf{A}^{(b)} = \sigma(\mathbf{Conv}^{(b)}) \in \mathbb{R}^{N_s \times C_b \times L_b^{\text{conv}}},$$

Nếu sử dụng chuẩn hóa theo lô, trên mỗi kênh f thực hiện

$$\begin{aligned} \mu_f &= \frac{1}{N_s L_b} \sum_{i,j} \mathbf{A}_{i,f,j}^{(b)} \\ \sigma_f^2 &= \frac{1}{N_s L_b} \sum_{i,j} \left(\mathbf{A}_{i,f,j}^{(b)} - \mu_f \right)^2, \\ \hat{\mathbf{A}}_{i,f,j}^{(b)} &= \frac{\mathbf{A}_{i,f,j}^{(b)} - \mu_f}{\sqrt{\sigma_f^2 + \varepsilon}} \\ \tilde{\mathbf{A}}_{i,f,j}^{(b)} &= \alpha_f \hat{\mathbf{A}}_{i,f,j}^{(b)} + \beta_f \end{aligned}$$

với α_f, β_f là các hệ số có thể học, đầu ra là $\tilde{\mathbf{A}}^{(b)} \in \mathbb{R}^{N_s \times C_b \times L_b^{\text{conv}}}$.

Nếu sử dụng ngắt bỏ ngẫu nhiên với xác suất giữ $1 - p_d$, trước hết tạo mặt nạ tuân theo phân phối Bernoulli và nhân Hadamard với đầu ra của lớp trước đó.

$$\mathbf{M}_{i,f,j} \sim \text{Bernoulli}(1 - p_d), \quad \bar{\mathbf{A}}_{i,f,j} = \frac{\tilde{\mathbf{A}}_{i,f,j}^{(b)} \mathbf{M}_{i,f,j}}{1 - p_d},$$

trong đó $\bar{\mathbf{A}}_{i,f,j} \in \mathbb{R}^{N_s \times C_b \times L_b^{\text{conv}}}$.

Kết thúc một khối là lớp gộp với phép toán pooling nhằm tổng hợp thông tin của đặc trưng, chẳng hạn gộp tối đại (max pooling) với kích thước bộ lọc K'_b , bước nhảy S'_b như sau

$$\mathbf{Pool}_{i,f,j}^{(b)} = \max_{0 \leq k \leq K'_b} \bar{\mathbf{A}}_{i,f,jS'_b+k},$$

hoặc gộp trung bình (average pooling) như sau

$$\mathbf{Pool}_{i,f,j}^{(b)} = \frac{1}{K'_b} \sum_{k=0}^{K'_b-1} \bar{\mathbf{A}}_{i,f,jS'_b+k},$$

có chiều dài

$$L_b^{\text{pool}} = \left\lfloor \frac{L_b^{\text{conv}} - K'_b}{S'_b} \right\rfloor + 1,$$

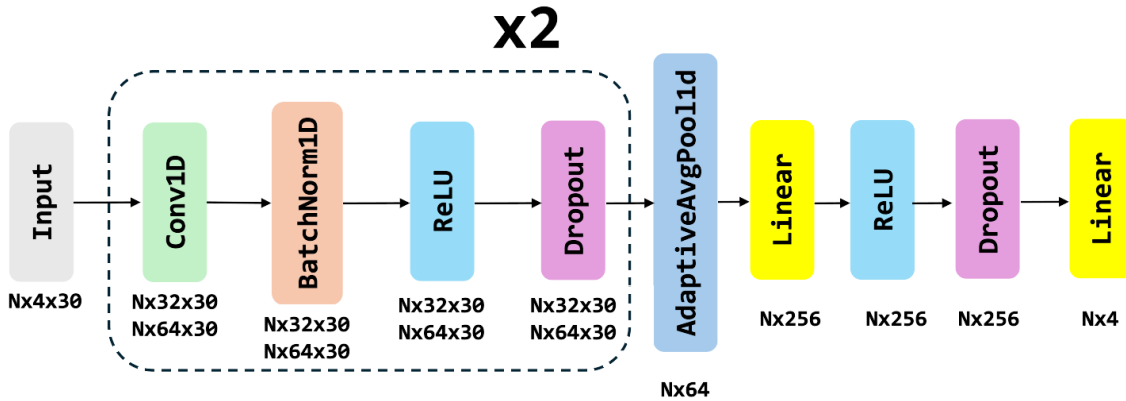
và $L_b^{\text{pool}} = L_b$ và $\mathbf{Pool}^{(b)} = \mathbf{X}^{(b)} \in \mathbb{R}^{N_s \times C_b \times L_b}$ là đầu ra của khối b .

Kết thúc B khối, thu được $\mathbf{X}^{(B)} \in \mathbb{R}^{N_s \times C_B \times L_B}$ và tái định hình thông qua việc trải phẳng để trở thành $\mathbf{z} \in \mathbb{R}^{N_s \times (C_B L_B)}$. Phần cuối của mạng sử dụng lớp dense (fully-connected) như một mạng truyền thẳng thông thường. Có tổng cộng M lớp dense như sau

$$\mathbf{h}^{(0)} = \mathbf{z}, \quad \mathbf{h}^{(m)} = \sigma \left(\mathbf{h}^{(m-1)} \mathbf{w}'^{(m)} + \mathbf{1}_{N_s} \mathbf{b}'^{(m)} \right), m = \overline{1, M-1}$$

và lớp cuối cùng không có hàm kích hoạt, lớp này đưa đầu ra trở thành đúng kích thước mong muốn

$$\hat{\mathbf{y}} = \mathbf{h}^{(M-1)} \mathbf{w}'^{(M)} + \mathbf{1}_{N_s} \mathbf{b}'^{(M)} \in \mathbb{R}^{N_s \times N_t}.$$



Hình 3: Các lớp trong TCN được triển khai trong báo cáo này

e. Mô hình mạng hồi tiếp

Mạng hồi tiếp (Recurrent Neural Network - RNN) phù hợp cho dữ liệu tuần tự (chuỗi thời gian, văn bản, tín hiệu...). Thay vì xử lý toàn bộ đầu vào độc lập, RNN có “bộ nhớ” (hidden state) để ghi nhận thông tin từ các bước trước.

(i) Kiến trúc Vanilla RNN

Trong bài toán dự đoán one-step, mạng RNN được thiết kế theo dạng many-to-one: sử dụng thông tin của p ngày lịch sử để cho ra 1 ngày tiếp theo. Các cách thiết kế khác không được đề cập trong báo cáo này.

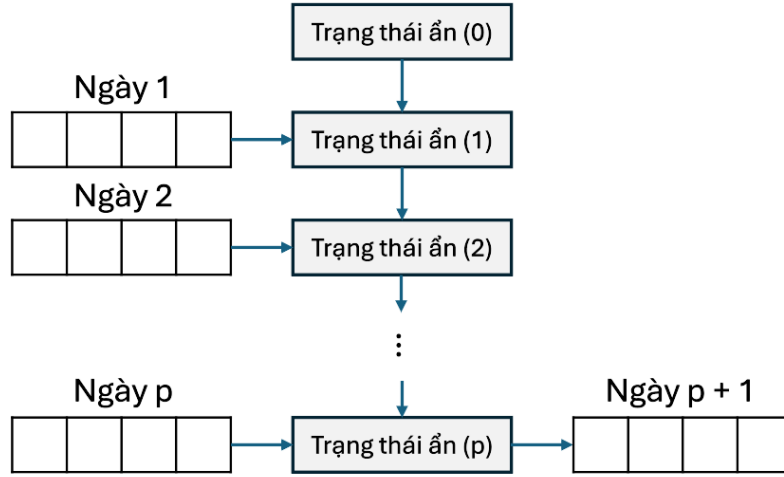
Với mạng RNN, ta tổ chức lại dữ liệu huấn luyện theo dạng

$$\left\{ \left(\mathbf{u}^{(i)}, \mathbf{y}^{(i)} \right) : i = \overline{1, N_s} \right\}$$

trong đó

$$\begin{aligned} \mathbf{u}^{(i)} &= (\mathbf{x}_i, \mathbf{x}_{i+1}, \dots, \mathbf{x}_{i+p-1}) \\ \mathbf{y}^{(i)} &= \mathbf{x}_{i+p}, \end{aligned}$$

với $\mathcal{D} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T\} \subset \mathbb{R}^{d \times 1}$ là dữ liệu quan sát được tại T mốc thời gian như đã đề cập. Triển khai trong PyTorch đảm bảo đầu vào là tensor với kích thước (batch_size, window_size, n_dimensions), khác thứ tự so với mạng tích chập.



Hình 4: Kiến trúc Vanilla RNN

Với mỗi mẫu dữ liệu i , trong p mốc thời gian, mỗi mốc j tương ứng có một trạng thái ẩn $\mathbf{h}_j^{(i)}$. Toàn bộ mạng RNN tạo thành bằng

$$\begin{aligned} \mathbf{h}_0^{(i)} &= \mathbf{0} \in \mathbb{R}^{h \times 1} \\ \mathbf{h}_t^{(i)} &= \varphi \left(\mathbf{W}_{uh} \mathbf{u}_t^{(i)} + \mathbf{W}_{hh} \mathbf{h}_{t-1}^{(i)} + \mathbf{b}_h \right), t = \overline{1, p} \\ \hat{\mathbf{y}}^{(i)} &= \mathbf{W}_{hy} \mathbf{h}_p^{(i)} + \mathbf{b}_y \end{aligned}$$

trong đó các ma trận và bias cần học là $\mathbf{W}_{uh} \in \mathbb{R}^{h \times d}$, $\mathbf{W}_{hh} \in \mathbb{R}^{h \times h}$, $\mathbf{W}_{hy} \in \mathbb{R}^{d \times h}$, $\mathbf{b}_h \in \mathbb{R}^{h \times 1}$, $\mathbf{b}_y \in \mathbb{R}^{d \times 1}$.

Một cách tổng quát, nếu ta đặt

$$f(\mathbf{h}, \mathbf{u}) = \varphi(\mathbf{W}_{uh} \mathbf{u} + \mathbf{W}_{hh} \mathbf{h} + \mathbf{b}_h)$$

thì thực chất mạng RNN liên tục lặp lại phép tổng hợp thông tin

$$\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{u}_t) = f(f(\mathbf{h}_{t-2}, \mathbf{u}_{t-1}), \mathbf{u}_t) = (f_{\mathbf{u}_t} \circ f_{\mathbf{u}_{t-1}} \circ \dots \circ f_{\mathbf{u}_1})(\mathbf{h}_0)$$

với mỗi $f_{\mathbf{u}_k}(\mathbf{h}) = f(\mathbf{h}, \mathbf{u}_k)$ nhằm gộp thông tin của $\mathbf{u}_k$ vào trạng thái ẩn $\mathbf{h}$. Nhờ vậy $\mathbf{h}_t$ tích lũy được thông tin của chuỗi $\mathbf{u}_1, \mathbf{u}_2, \dots, \mathbf{u}_t$.

(ii) Kiến trúc LSTM

LSTM (Long Short-Term Memory) mở rộng RNN bằng cách duy trì thêm “cell state” $\mathbf{c}_t$, cho phép mạng chọn lọc giữ lại hay quên thông tin dài hạn. Cụ thể, tại mỗi bước t , với đầu vào $\mathbf{u}_t^{(i)} \in \mathbb{R}^{d \times 1}$ và trạng thái ẩn trước đó $\mathbf{h}_{t-1}^{(i)} \in \mathbb{R}^{h \times 1}$, LSTM thực hiện:

1. Cổng nhập (input gate) $\mathbf{i}_t^{(i)}$ quyết định bao nhiêu thông tin mới từ $\mathbf{u}_t$ được ghi vào cell state. Ở đây hàm kích hoạt sigmoid $\sigma(\cdot) \in (0, 1)$ đóng vai trò như một “van” định lượng, giúp mạng tự học cách mở (giá trị gần 1) hoặc đóng (giá trị gần 0) luồng thông tin từ đầu vào ở mỗi thời điểm.

$$\mathbf{i}_t^{(i)} = \sigma(\mathbf{W}_{ui} \mathbf{u}_t^{(i)} + \mathbf{W}_{ui} \mathbf{h}_{t-1}^{(i)} + \mathbf{b}_i)$$

2. Cổng quên (forget gate) $\mathbf{f}_t^{(i)}$ điều khiển mức độ giữ lại thông tin cũ trong $\mathbf{c}_{t-1}$. Khi $\mathbf{f}_t$ gần 1, ký ức trước đó được bảo lưu; khi gần 0, nó dần bị xoá đi. Nhờ vậy mạng có thể bỏ qua những tín hiệu không còn hữu ích, giảm hiện tượng “quá tải” thông tin.

$$\mathbf{f}_t^{(i)} = \sigma(\mathbf{W}_{uf} \mathbf{u}_t^{(i)} + \mathbf{W}_{hf} \mathbf{h}_{t-1}^{(i)} + \mathbf{b}_f)$$

3. Cổng tạo ứng viên (cell candidate) $\tilde{\mathbf{c}}_t^{(i)}$ sinh ra thông tin mới tiềm năng để cập nhật vào cell state. Hàm tanh($\cdot$) chuẩn hoá ứng viên vào $(-1, 1)$, cho phép mạng biểu diễn cả tác động dương và âm tới bộ nhớ.

$$\tilde{\mathbf{c}}_t^{(i)} = \tanh(\mathbf{W}_{ug} \mathbf{u}_t^{(i)} + \mathbf{W}_{hg} \mathbf{h}_{t-1}^{(i)} + \mathbf{b}_g)$$

4. Cập nhật cell state. Cổng quên $\mathbf{f}_t^{(i)}$ xác định phần ký ức cũ giữ lại, trong khi cổng nhập $\mathbf{i}_t^{(i)}$ chọn lọc bao nhiêu thông tin mới $\tilde{\mathbf{c}}_t^{(i)}$ sẽ được cộng vào. Sự kết hợp hai tác nhân này tạo nên cơ chế “ghi-xoá” linh hoạt, cho phép LSTM lưu trữ các mẫu phụ thuộc dài hạn.

$$\mathbf{c}_t^{(i)} = \mathbf{f}_t^{(i)} \odot \mathbf{c}_{t-1}^{(i)} + \mathbf{i}_t^{(i)} \odot \tilde{\mathbf{c}}_t^{(i)}$$

trong đó $\odot$ là nhân Hadamard, thực hiện trên từng phần tử.

5. Cổng xuất (output gate) $\mathbf{o}_t^{(i)}$ quyết định bao nhiêu phần của cell state mới được “phát lộ” ra ngoài làm hidden state. Điều này giúp bóc tách thông tin phục vụ tác vụ dự đoán trong khi vẫn giữ kín phần ký ức cần thiết bên trong $\mathbf{c}_t$.

$$\mathbf{o}_t^{(i)} = \sigma(\mathbf{W}_{uo} \mathbf{u}_t^{(i)} + \mathbf{W}_{ho} \mathbf{h}_{t-1}^{(i)} + \mathbf{b}_o)$$

6. Trạng thái ẩn (hidden state). Sau khi điều tiết bởi cổng xuất, giá trị $\tanh(\mathbf{c}_t^{(i)})$ (phiên bản “nén” của cell state) được nhân Hadamard với $\mathbf{o}_t^{(i)}$ để tạo ra $\mathbf{h}_t^{(i)}$. Hidden state này vừa phản ánh thông tin ngắn hạn cần đưa ra mạng kế tiếp, vừa duy trì tính liên kết với bộ nhớ dài hạn thông qua $\mathbf{c}_t^{(i)}$.

$$\mathbf{h}_t^{(i)} = \mathbf{o}_t^{(i)} \odot \tanh(\mathbf{c}_t^{(i)})$$

Cuối cùng, với p bước thời gian, ta sử dụng $\mathbf{h}_p^{(i)}$ để dự đoán one-step:

$$\hat{\mathbf{y}}^{(i)} = \mathbf{W}_{hy} \mathbf{h}_p^{(i)} + \mathbf{b}_y,$$

với $\mathbf{W}_{u\{i,f,g,o\}} \in \mathbb{R}^{h \times d}$, $\mathbf{W}_{h\{i,f,g,o\}} \in \mathbb{R}^{h \times h}$, $\mathbf{b}_{\{i,f,g,o\}} \in \mathbb{R}^{h \times 1}$, $\mathbf{W}_{hy} \in \mathbb{R}^{d \times h}$, $\mathbf{b}_y \in \mathbb{R}^{d \times 1}$ là các tham số cần được học.

f. Mô hình mạng biến hình

Mạng biến hình (Transformer) được giới thiệu lần đầu trong lĩnh vực xử lý ngôn ngữ tự nhiên, nổi bật với cơ chế “chú ý” (attention) giúp mô hình tập trung vào các phần thông tin quan trọng trong chuỗi đầu vào. Không giống như các mạng RNN hay LSTM truyền thống vốn xử lý tuần tự, mạng biến hình xử lý toàn bộ chuỗi đồng thời, cho phép học được các mối quan hệ dài hạn một cách hiệu quả hơn.

Mạng biến hình là nền tảng cho các mô hình ngôn ngữ lớn (Large Language Model - LLM) hiện đang dần trở nên phổ biến. Một kiến trúc thành công như vậy là cảm hứng để chúng ta thử nghiệm hiệu quả của nó trên nhiệm vụ dự báo chuỗi thời gian.

(i) Kiến trúc TFT

Để phân biệt mô hình Transformer được dùng cho xử lý ngôn ngữ tự nhiên, thì mạng biến hình cho nhiệm vụ dự báo chuỗi thời gian có tên gọi là *mạng biến hình tổ hợp thời gian* (Temporal Fusion Transformer - TFT). Báo cáo này triển khai một phiên bản rất đơn giản của TFT, để minh họa các cơ chế nổi bật của mạng.

Đầu tiên, dữ liệu huấn luyện được chuẩn bị giống như Vanilla RNN đã đề cập ở phần trước đó. Với một mẫu dữ liệu i , và mốc t trong p mốc thời gian, tức là $\mathbf{u}_t^{(i)} \in \mathbb{R}^d$. Ta dùng một lớp tuyến tính để đưa mốc thời gian này từ không gian $\mathbb{R}^d$ lên không gian $\mathbb{R}^{d'}$

$$\mathbf{e}_t^{(i)} = \mathbf{u}_t^{(i)} \mathbf{W}_e + \mathbf{b}_e \in \mathbb{R}^{1 \times d'}, \quad \mathbf{W}_e \in \mathbb{R}^{d' \times d'}$$

nhằm nhúng biểu diễn thô của dữ liệu vào một không gian tiềm ẩn (latent space) giàu biểu diễn hơn.

Qua bước này, ta đã chuyển $\mathbf{u}^{(i)} \rightarrow \mathbf{e}^{(i)} = [\mathbf{e}_1^{(i)}, \mathbf{e}_2^{(i)}, \dots, \mathbf{e}_p^{(i)}] \in \mathbb{R}^{p \times d'}$. Để thực hiện cơ chế chú ý, chúng ta biến đổi vector nhúng cuối cùng và toàn bộ biểu diễn nhúng lịch sử thành các ma trận truy vấn, khóa và giá trị:

$$\begin{aligned} \mathbf{Q} &= \mathbf{e}_p^{(i)} \mathbf{W}_q \in \mathbb{R}^{1 \times d'}, & \mathbf{W}_q &\in \mathbb{R}^{d' \times d'}, \\ \mathbf{K} &= \mathbf{E}^{(i)} \mathbf{W}_k \in \mathbb{R}^{p \times d'}, & \mathbf{W}_k &\in \mathbb{R}^{d' \times d'}, \\ \mathbf{V} &= \mathbf{E}^{(i)} \mathbf{W}_v \in \mathbb{R}^{p \times d'}, & \mathbf{W}_v &\in \mathbb{R}^{d' \times d'}. \end{aligned}$$

Trong đó, $\mathbf{Q}$ (query) phản ánh “câu hỏi” mà mô hình đặt ra dựa trên trạng thái cuối cùng $\mathbf{e}_p^{(i)}$, $\mathbf{K}$ (key) là “chỉ mục” được sinh ra từ toàn bộ chuỗi embedding để xác định mức độ liên quan, và $\mathbf{V}$ (value) chứa thông tin chi tiết mà mạng sẽ tập hợp dựa trên trọng số chú ý. Quá trình này tương tự cơ chế tìm kiếm trong cơ sở dữ liệu: query giống như một từ khóa truy vấn, keys như các chỉ mục chỉ ra vị trí dữ liệu tương tác, values như bản ghi chứa nội dung.

Để tính điểm chú ý, ta thực hiện phép nhân ma trận đã chuẩn hóa:

$$\text{scores} = \frac{\mathbf{Q} \mathbf{K}^\top}{\sqrt{d'}} \in \mathbb{R}^{1 \times p}.$$

Phép toán $\mathbf{Q} \mathbf{K}^\top$ biểu diễn tích vô hướng giữa truy vấn và từng khóa, tổng hợp mức độ tương quan qua các chiều d' . Việc chia cho $\sqrt{d'}$ không chỉ ổn định phân phối score mà còn hạn chế gradient bão hòa khi d' tăng cao.

Kết quả scores sau đó được đưa qua hàm softmax để chuyển thành xác suất chú ý $\alpha \in \mathbb{R}^{1 \times p}$. Bằng cách này, mạng tự động điều chỉnh trọng số α_t cho mỗi bước lịch sử sao cho tổng bằng 1, đồng thời phục vụ việc tập trung vào những bước quan trọng nhất.

Cuối cùng, vector ngữ cảnh $\mathbf{c} \in \mathbb{R}^{d'}$ được tạo ra bằng tích có trọng số giữa ma trận giá trị và xác suất chú ý:

$$\mathbf{c} = \mathbf{V}^{(i)} \alpha^\top = \sum_{t=1}^p \alpha_t \mathbf{v}_t,$$

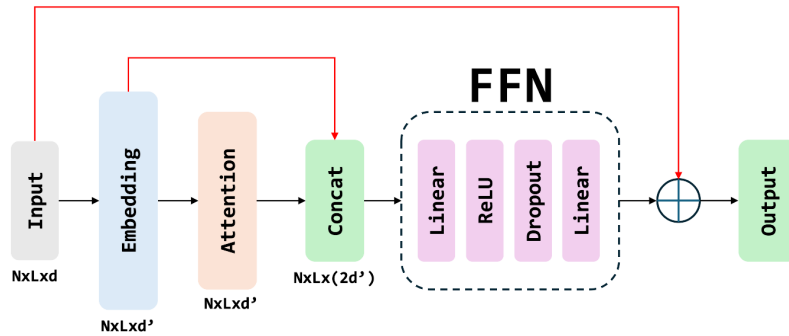
trong đó $\mathbf{v}_t$ là cột thứ t của $\mathbf{V}$. Vector này hội tụ thông tin lịch sử có liên quan trực tiếp đến yêu cầu dự đoán bước kế, sau đó được ghép nối với vector nhúng tại mốc thời gian cuối cùng

$$\mathbf{r} = [\mathbf{c}^{(i)} \mid \mathbf{e}_p^{(i)}] \in \mathbb{R}^{2d'}$$

nhằm đảm bảo thông tin gốc luôn đi thẳng đến hỗ trợ cho phần dự báo.

Sau đó, mô hình sử dụng một mạng truyền thẳng đơn giản, gọi là FFN, để dự báo $\delta = \text{FFN}(\mathbf{r})$ so với kết quả cuối cùng $\hat{\mathbf{y}}^{(i)} = \mathbf{u}_p^{(i)} + \delta$, giảm mục tiêu học xuống dự báo sai phân $\Delta \mathbf{x} = \mathbf{x}_{t+1} - \mathbf{x}_t$.

Có nhiều lý do để áp dụng phương pháp này. Trước hết, chuỗi thời gian giá thường có xu hướng không dừng (non-stationary), tức là mức giá tuyệt đối có thể lên xuống và vượt ngoài giới hạn đã biết khiến việc dự báo gặp khó khăn. Trong khi đó, sai phân bậc 1 của chuỗi có tính ổn định hơn, và thậm chí có thể đạt được tính dừng. Một lý do khác nữa, phương pháp này lấy cảm hứng từ một kiến trúc đặc biệt thành công trong thị giác máy tính là ResNet, trong đó sử dụng *học thặng dư* (residual learning), đã được chứng minh là mang lại hiệu quả cao trong đào tạo mô hình mạng phức tạp.



Hình 5: Kiến trúc Transformer

3.2 Mô hình xác suất

(i) Kiến trúc VAE

Báo cáo này giới thiệu mô hình *mạng biến phân tự mã hóa* (Variational AutoEncoder - VAE) cho bài toán dự báo phân phối, tuy nhiên không được thực nghiệm. Triển khai thông thường nhất, VAE bao gồm mạng mã hóa (f_ϕ) để nhúng thông tin vào không gian tiềm ẩn và mạng giải mã (g_θ) chuyển đổi vector tiềm ẩn trở về không gian gốc.

Nhắc lại từ những phần trước, chúng ta đã có một chuỗi quan sát lịch sử

$$\mathbf{x}_{t-p+1:t} = (\mathbf{x}_{t-p+1}, \mathbf{x}_{t-p+2}, \dots, \mathbf{x}_t), \quad \mathbf{x}_i \in \mathbb{R}^d,$$

và chúng ta muốn dự đoán phân phối của $\mathbf{x}_{t+1:t+H}$.

Mạng mã hóa f_ϕ chuyển chuỗi lịch sử thành tham số của phân phối tiềm ẩn:

$$\begin{aligned} \mathbf{h}_e &= f_\phi(\mathbf{x}_{t-p+1:t}), & \mathbf{h}_e &\in \mathbb{R}^{H_e}, \\ \boldsymbol{\mu}_z &= \mathbf{W}_\mu^{\text{enc}} \mathbf{h}_e + \mathbf{b}_\mu^{\text{enc}}, & \boldsymbol{\mu}_z &\in \mathbb{R}^Z, \\ \log \boldsymbol{\sigma}_z^2 &= \mathbf{W}_\sigma^{\text{enc}} \mathbf{h}_e + \mathbf{b}_\sigma^{\text{enc}}, & \boldsymbol{\sigma}_z &\in \mathbb{R}^Z, \\ z &\sim \mathcal{N}(\boldsymbol{\mu}_z, \text{diag}(\boldsymbol{\sigma}_z^2)). \end{aligned}$$

Mạng giải mã g_θ lấy mẫu tiềm ẩn z và sinh phân phối cho tương lai:

$$\begin{aligned} \mathbf{h}_d &= g_\theta(z), \quad \mathbf{h}_d \in \mathbb{R}^{H_d}, \\ \boldsymbol{\mu}_x &= \mathbf{W}_\mu^{\text{dec}} \mathbf{h}_d + \mathbf{b}_\mu^{\text{dec}}, \quad \boldsymbol{\mu}_x \in \mathbb{R}^{H \times d}, \\ \log \boldsymbol{\sigma}_x^2 &= \mathbf{W}_\sigma^{\text{dec}} \mathbf{h}_d + \mathbf{b}_\sigma^{\text{dec}}, \quad \boldsymbol{\sigma}_x \in \mathbb{R}^{H \times d}, \\ \mathbf{x}_{t+1:t+H} &| z \sim \mathcal{N}(\boldsymbol{\mu}_x, \text{diag}(\boldsymbol{\sigma}_x^2)). \end{aligned}$$

Các tham số của mạng nhằm tối thiểu hóa hàm cận dưới:

$$\mathcal{L}(\theta, \phi) = \mathbb{E}_{q_\phi(z|\mathbf{x}_{t-L+1:t})} \left[\log p_\theta(\mathbf{x}_{t+1:t+H} | z) \right] - \text{KL}[q_\phi(z | \mathbf{x}_{t-L+1:t}) \parallel p(z)],$$

trong đó $p(z) = \mathcal{N}(0, I)$ và $\text{KL}[\cdot \parallel \cdot]$ là phân kỳ Kullback-Leibler.

4 Tập thể hóa dự báo

Phương pháp *học tập tập thể* (ensemble learning) là kỹ thuật kết hợp đồng thời nhiều mô hình học máy (gọi là các base learners) để tạo ra một mô hình tổng hợp (ensemble) có khả năng dự báo hoặc phân lớp chính xác và ổn định hơn so với bất kỳ mô hình đơn lẻ nào. Về cơ bản, ý tưởng chính của ensemble learning là tận dụng tính đa dạng (diversity) và khả năng bù trừ lẫn nhau (complementarity) giữa các base learners: khi một mô hình mắc sai lầm, những mô hình khác có thể “kéo” kết quả dự báo về gần với giá trị thật.

Những phương pháp ensemble phổ biến bao gồm:

- Bagging (Bootstrap Aggregating): huấn luyện song song nhiều mô hình trên các tập con khác nhau được lấy mẫu bootstrap từ dữ liệu gốc, rồi lấy trung bình (với bài toán hồi quy) hoặc bỏ phiếu đa số (với phân lớp).
- Boosting: xây dựng các mô hình một cách tuần tự, trong đó mỗi mô hình mới tập trung học trên những mẫu mà các mô hình trước đó dự đoán sai; sau khi huấn luyện xong, các dự báo của từng mô hình được kết hợp với nhau dưới dạng tổng trọng số để cho ra kết quả cuối cùng.
- Stacking (Stacked Generalization): tạo ra một lớp mô hình meta (meta-learner) để “học cách kết hợp” đầu ra của nhiều base learners, thường đạt hiệu quả cao nhưng yêu cầu cẩn thận trong việc chia tập và tránh rò rỉ dữ liệu.

Trong số các kỹ thuật tập thể hóa dự báo, Blending thường được xem là một biến thể đơn giản của Stacking. Thay vì sử dụng kỹ thuật chéo (k-fold cross-validation) phức tạp để sinh ra tập đặc trưng đầu vào cho meta-learner, blending chia dữ liệu huấn luyện thành hai phần:

- Tập con A (thường chiếm khoảng 70–80%): dùng để huấn luyện tất cả các base learners.
- Tập hold-out B (phần còn lại): dùng để dự báo bằng các base learners đã huấn luyện, tạo thành tập dữ liệu mới gồm các cột là đầu ra dự báo của từng base learner và nhãn thật.

Sau đó, ta huấn luyện một mô hình tổng hợp (blender/meta-learner) trên tập B này, học cách ghép các dự báo của base learners thành dự báo cuối cùng. Khi áp dụng cho dữ liệu mới, các base learners đầu tiên sẽ đưa ra dự báo, sau đó blender sẽ kết hợp chúng để cho ra kết quả cuối cùng. Blending thường đơn giản về triển khai và tính toán, tuy nhiên cần chú ý lựa chọn tỷ lệ chia A/B sao cho đủ lớn để đảm bảo cả base learners và blender đều được huấn luyện tốt, đồng thời tránh hiện tượng overfitting trên tập hold-out.

Ngoài ra, để củng cố về mặt lý thuyết rằng việc áp dụng ensemble learning có thể giảm thiểu lỗi và tăng hiệu quả của mô hình, chúng tôi đưa ra chứng minh dựa trên phân rã *bias-variance*. Thật vậy, xét bài toán hồi quy với dữ liệu (x, y) sinh ra từ một phân phối chung và bộ dữ liệu huấn luyện $\mathcal{D}$ gồm n quan sát. Giả sử mô hình học được là $\hat{f}(x; \mathcal{D})$. Ta quan tâm đến độ sai trung bình bình phương (Mean Squared Error) tại điểm x :

$$\text{MSE}(x) = \mathbb{E}_{\mathcal{D}, y} \left[\left(\hat{f}(x; \mathcal{D}) - y \right)^2 \right].$$

Phân rã thành ba thành phần:

$$\text{MSE}(x) = \underbrace{\left(\mathbb{E}_{\mathcal{D}}[\hat{f}(x; \mathcal{D})] - f^*(x) \right)^2}_{\text{Bias}^2} + \underbrace{\mathbb{E}_{\mathcal{D}} \left[\left(\hat{f}(x; \mathcal{D}) - \mathbb{E}_{\mathcal{D}}[\hat{f}(x; \mathcal{D})] \right)^2 \right]}_{\text{Variance}} + \underbrace{\mathbb{E}[(y - f^*(x))^2]}_{\sigma^2},$$

trong đó $f^*(x) = \mathbb{E}[y | x]$ là hàm mục tiêu lý tưởng và σ^2 là phương sai nhiễu.

Khi kết hợp M mô hình base learners $\{\hat{f}_m\}_{m=1}^M$ theo tổ hợp lồi

$$\hat{f}_{\text{ens}}(x) = \sum_{m=1}^M w_m \hat{f}_m(x), \quad \sum_{m=1}^M w_m = 1, \quad w_m \geq 0,$$

thì

$$\text{Bias}^2(\hat{f}_{\text{ens}}) = \left(\mathbb{E}[\hat{f}_{\text{ens}}] - f^* \right)^2 = \left(\sum_m w_m \mathbb{E}[\hat{f}_m] - f^* \right)^2 \leq \sum_m w_m \text{Bias}^2(\hat{f}_m), \quad (1)$$

$$\text{Var}(\hat{f}_{\text{ens}}) = \sum_m w_m^2 \text{Var}(\hat{f}_m) + \sum_{m \neq m'} w_m w_{m'} \text{Cov}(\hat{f}_m, \hat{f}_{m'}). \quad (2)$$

Nếu các $\hat{f}_m$ gần như không tương quan, với trọng số đồng đều $w_m = 1/M$, ta có

$$\text{Var}(\hat{f}_{\text{ens}}) \approx \frac{1}{M^2} \sum_{m=1}^M \text{Var}(\hat{f}_m) = \frac{1}{M} \overline{\text{Var}},$$

tức Variance giảm tỉ lệ $1/M$, trong khi Bias² không tăng, chứng tỏ rằng việc kết hợp các mô hình đơn lẻ theo tổ hợp lồi đạt được kết quả ít nhất không tệ hơn một mô hình đơn lẻ.

5 Thực nghiệm so sánh

5.1 Mô tả dữ liệu

a. Thu thập dữ liệu

Đảm bảo tính mới, tính ứng dụng và tính khách quan, báo cáo này sử dụng tập dữ liệu là giá cổ phiếu của 30 công ty niêm yết trên sàn chứng khoán Việt Nam, thuộc VN30 - nhóm các công ty có vốn hóa và thanh khoản lớn nhất thị trường. Dữ liệu về giá thường được xem như rất khó để nắm bắt quy luật, đặt ra thách thức đối với chúng tôi trong việc xây dựng mô hình.

Dữ liệu được thu thập thông qua thư viện `vnstock` dưới dạng các bảng ứng với mỗi mã, bao gồm các cột sau đây:

- `time` là mốc thời gian.
- `open` là giá mở cửa của cổ phiếu.
- `high` là giá cao nhất trong ngày.
- `low` là giá thấp nhất trong ngày.
- `close` là giá đóng cửa của cổ phiếu.

trong đó, `time` lấy đến tần suất ngày, đã sắp xếp thứ tự. Những ngày xuất hiện trong bảng tương ứng là những ngày sản mở cửa giao dịch, những ngày không tồn tại trong cột tức là sàn nghỉ giao dịch (do nghỉ lễ, nghỉ Tết...). Chúng tôi xem như những ngày tồn tại trong bảng là ngày làm việc liên tiếp, và không điền giá trị của các ngày thiếu. Mục tiêu dự báo cho một ngày làm việc kế tiếp, không phải một ngày liên tục theo thời gian.

Về kích thước các tập dữ liệu, cụ thể

- **Tập huấn luyện** là khoảng thời gian từ 01/01/2019 đến 31/12/2023 (tương ứng với 5 năm), gồm khoảng 1200 ngày giao dịch chính thức.
- **Tập kiểm tra** là khoảng thời gian từ 01/01/2024 đến 30/4/2025 (tương ứng với 1.5 năm), gồm khoảng 300 ngày giao dịch chính thức.

b. Tiền xử lý dữ liệu

Cả 2 tập dữ liệu huấn luyện và kiểm tra được chuẩn hóa bằng kỳ vọng và phương sai của tập huấn luyện để đảm bảo không có data leakage.

Sau đó, chúng tôi tiến hành tạo các đặc trưng trễ (lag feature) gồm 30 ngày lịch sử, mỗi ngày là một vector 4 chiều, mục tiêu dự đoán cũng là một vector 4 chiều, gồm đồng thời `open`, `high`, `low`, `close`. Như vậy kích thước của tập huấn luyện gồm: đặc trưng có dạng $(N_s, 120)$ và mục tiêu có dạng $(N_s, 4)$.

Trong trường hợp đánh giá ảnh hưởng của đặc trưng, chúng tôi có thể làm giàu vector đặc trưng bằng cách bổ sung thêm các thông tin, chẳng hạn về ngày cần dự đoán ngoài, thuộc tính lịch sử. Điều này là hoàn toàn biết được vào thời điểm dự đoán: ta luôn biết đầy đủ thông tin về ngày sẽ thực hiện dự đoán, bao gồm cả chính ngày đó (như thứ của ngày đó trong tuần, ngày đó trong tháng, tuần của ngày đó trong năm...).

Tóm lại, các nhóm đặc trưng bao gồm:

1. Đặc trưng trễ (lag features):

Với mỗi biến $x \in \{\text{open, high, low, close}\}$ và độ trễ $i = 1, \dots, L$, ta có

$$\text{feat_lag}_{t,i} = x_{t-i}.$$

Các đặc trưng này giúp mô hình nắm được phụ thuộc thời gian bằng cách đưa vào giá trị quá khứ của từng biến.

2. Đặc trưng lịch (calendar features):

Mã hóa tuần, ngày, tháng, tuần trong năm dưới dạng sin-cos:

$$\begin{aligned} \text{dow_sin}_t &= \sin\left(2\pi \frac{\text{weekday}(t)}{7}\right), & \text{dow_cos}_t &= \cos\left(2\pi \frac{\text{weekday}(t)}{7}\right), \\ \text{dom_sin}_t &= \sin\left(2\pi \frac{\text{day}(t)-1}{31}\right), & \text{dom_cos}_t &= \cos\left(2\pi \frac{\text{day}(t)-1}{31}\right), \\ \text{month_sin}_t &= \sin\left(2\pi \frac{\text{month}(t)-1}{12}\right), & \text{month_cos}_t &= \cos\left(2\pi \frac{\text{month}(t)-1}{12}\right), \\ \text{woy_sin}_t &= \sin\left(2\pi \frac{\text{week}(t)-1}{52}\right), & \text{woy_cos}_t &= \cos\left(2\pi \frac{\text{week}(t)-1}{52}\right). \end{aligned}$$

Các mã hóa sin-cos này giữ lại tính chu kỳ của lịch (tuần, tháng, ...) mà không tạo ra đứt gãy khi chuyển giao giữa cuối-đầu chu kỳ.

3. Đặc trưng trượt (rolling statistics):

Trung bình, độ lệch chuẩn, giá trị nhỏ nhất và lớn nhất trên cửa sổ L , shift 1 bước:

$$\mu_t = \frac{1}{L} \sum_{i=1}^L x_{t-i}, \quad \sigma_t = \sqrt{\frac{1}{L-1} \sum_{i=1}^L (x_{t-i} - \mu_t)^2}, \quad x_t^{\min} = \min_{1 \leq i \leq L} x_{t-i}, \quad x_t^{\max} = \max_{1 \leq i \leq L} x_{t-i}.$$

Các thống kê trượt này cung cấp thông tin về xu hướng và biên độ biến động của biến trong khoảng quá khứ.

4. Chỉ báo kỹ thuật (technical indicators):

- **SMA_n:**

$$\text{SMA}_{n,t} = \frac{1}{n} \sum_{j=0}^{n-1} P_{t-j}.$$

trong đó, P_t là giá đóng cửa tại thời điểm t .

- **EMA_n:**

$$\text{EMA}_{n,t} = \alpha P_t + (1 - \alpha) \text{EMA}_{n,t-1}, \quad \alpha = \frac{2}{n+1}.$$

- **MACD:**

$$\begin{aligned} \text{MACD}_t &= \text{EMA}_{12,t} - \text{EMA}_{26,t}, \\ \text{Signal}_t &= \text{EMA}_9(\text{MACD}). \end{aligned}$$

- **RSI₁₄:**

$$\begin{aligned} \text{RSI}_{14,t} &= 100 - \frac{100}{1 + \frac{\bar{g}_t}{\bar{\ell}_t}}, \\ \bar{g}_t &= \frac{1}{14} \sum_{j=1}^{14} g_{t-j}, \quad \bar{\ell}_t = \frac{1}{14} \sum_{j=1}^{14} \ell_{t-j}. \end{aligned}$$

- **ATR₁₄:**

$$\text{TR}_t = \max\{H_t - L_t, |H_t - P_{t-1}|, |L_t - P_{t-1}|\},$$

$$\text{ATR}_{14,t} = \frac{1}{14} \sum_{j=1}^{14} \text{TR}_{t-j}.$$

với L_t, H_t là giá thấp, cao tại thời điểm t .

- **Bollinger Bands:**

$$\text{Upper}_t = \text{SMA}_{20,t} + 2\sigma_{20,t},$$

$$\text{Lower}_t = \text{SMA}_{20,t} - 2\sigma_{20,t}.$$

5. Đặc trưng phi tuyến (nonlinear features):

$$r_t = \ln\left(\frac{P_t}{P_{t-1}}\right), \quad \log_return_t = r_{t-1}, \quad \text{return_accel}_t = r_{t-1} - r_{t-2},$$

$$\text{return_skew}_t = \text{skew}(r_{t-L}, \dots, r_{t-1}), \quad \text{return_kurt}_t = \text{kurt}(r_{t-L}, \dots, r_{t-1}),$$

$$\text{price_pct_rank}_t = \frac{\#\{P_{t-i} < P_{t-1}\}}{L},$$

$$\text{fft_amp1}_t = |\text{FFT}_1(P_{t-L}, \dots, P_{t-1})|,$$

$$\text{fft_amp2}_t = |\text{FFT}_2(P_{t-L}, \dots, P_{t-1})|.$$

với FFT_k là biên độ của thành phần tần số thứ k trong biến đổi Fourier.

6. Đặc trưng tự tương quan (autocorrelation features):

Với r_t là log-return, autocorrelation tại lag $k \in \{1, 5, 10\}$ trên cửa sổ $L + 1$:

$$\text{acf}_{k,t} = \frac{\sum_{i=0}^L (r_{t-i} - \bar{r})(r_{t-i-k} - \bar{r})}{\sum_{i=0}^L (r_{t-i} - \bar{r})^2}.$$

nhằm đo lường mức độ phụ thuộc giữa các giá trị log-return cách nhau k ngày, giúp phát hiện mẫu chu kỳ và tự tương quan.

7. Đặc trưng xu hướng (trend slope):

Độ dốc của đường hồi quy bậc nhất trên các điểm (i, P_{t-L+i}) , $i = 0, \dots, L - 1$:

$$\text{trend_slope}_t = \frac{\sum_{i=0}^{L-1} (i - \bar{i})(P_{t-L+i} - \bar{P})}{\sum_{i=0}^{L-1} (i - \bar{i})^2},$$

trong đó $\bar{i} = \frac{L-1}{2}$ và $\bar{P} = \frac{1}{L} \sum_{i=0}^{L-1} P_{t-L+i}$.

c. Tính chất dữ liệu

1. *Tính tự tương quan.* Chúng tôi đã thực hiện kiểm chứng *Ljung-Box* về tính tự tương quan trên cả 30 mã, với cả 4 thuộc tính. Trên tất cả các mã và 4 chuỗi thời gian tương ứng, kết quả đều khẳng định có tính tự tương quan mạnh mẽ.

2. *Tính nhân quả.* Tiếp theo, chúng tôi thực hiện kiểm chứng nhân quả *Granger* trong cùng mỗi mã, giữa về nguyên nhân và về kết quả (2 chiều) lần lượt là **open**, **high**, **low**, **close** và nhận thấy chúng đều củng cố cho giả thuyết nhân quả giữa các chuỗi thời gian này. Chi tiết có thể xem tại mã nguồn.

3. *Tính dừng.* Chuỗi thời gian là dừng (stationary) nếu có các đặc tính thống kê không đổi theo thời gian. Trong phần này chúng tôi áp dụng kiểm chứng *Augmented Dickey-Fuller* và nhận thấy, hầu hết 4 chuỗi thời gian **open**, **high**, **low**, **close**, trên tất cả các mã cổ phiếu là không dừng, và dừng sau khi lấy sai phân bậc 1.

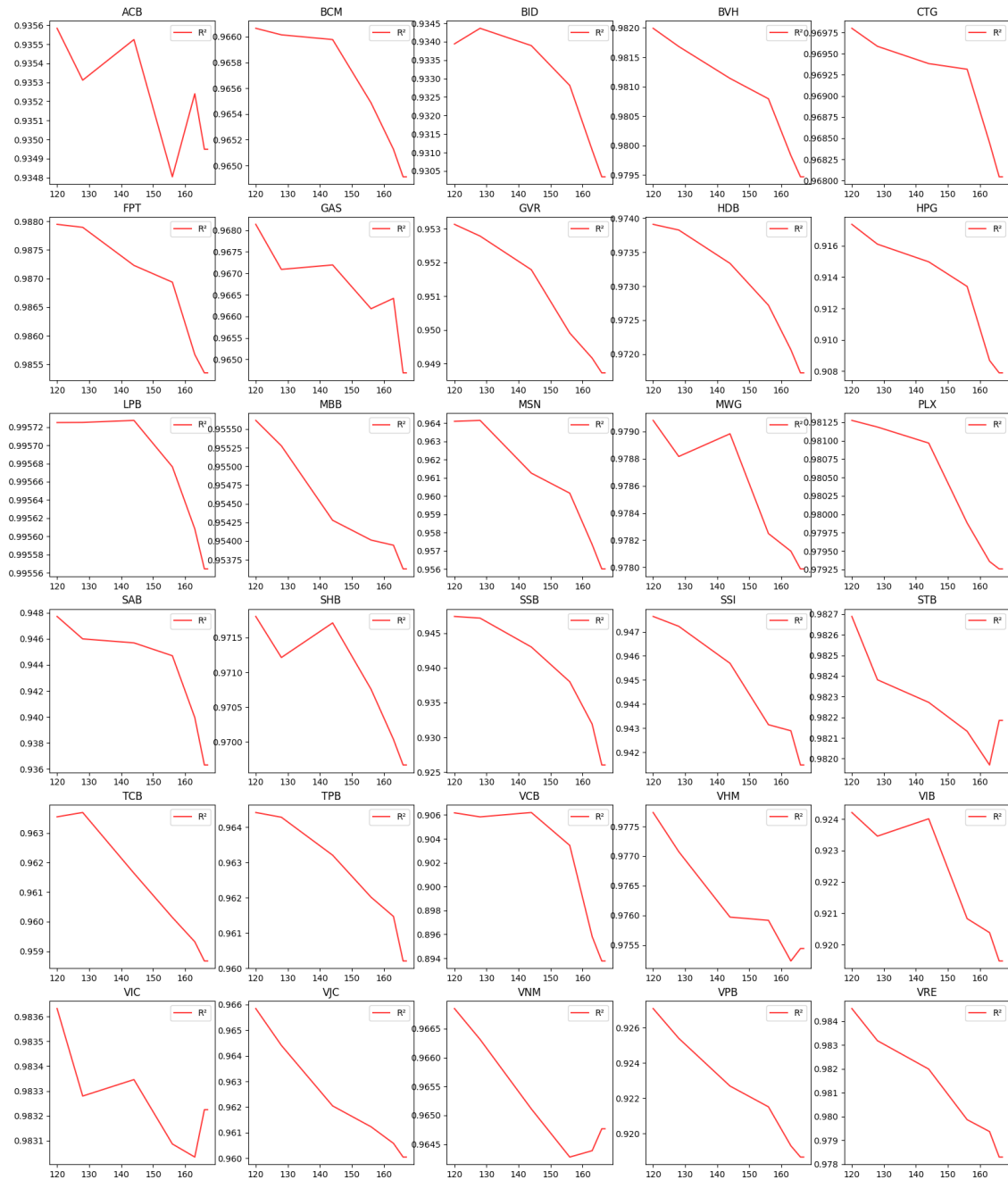
5.2 Mô hình học máy

Thực nghiệm 1. Ảnh hưởng của đặc trưng lên hiệu quả mô hình

Như đã đề cập trong phần tiền xử lý dữ liệu, có 6 nhóm đặc trưng, và chúng tôi xem xét việc bổ sung thêm đặc trưng “phức tạp” liệu có khiến dự báo tốt hơn không. Kết quả trình bày tại [Hình 6](#), cho thấy mặc dù đặc trưng tăng lên nhưng OLS có xu hướng kém đi. Rõ ràng mô hình OLS quản lý đặc trưng không hiệu quả. Ngoài ra, do tính tự tương quan mạnh của chuỗi mà những đặc trưng phi tuyến không hỗ trợ gì thêm, thậm chí gây ra nhiễu.

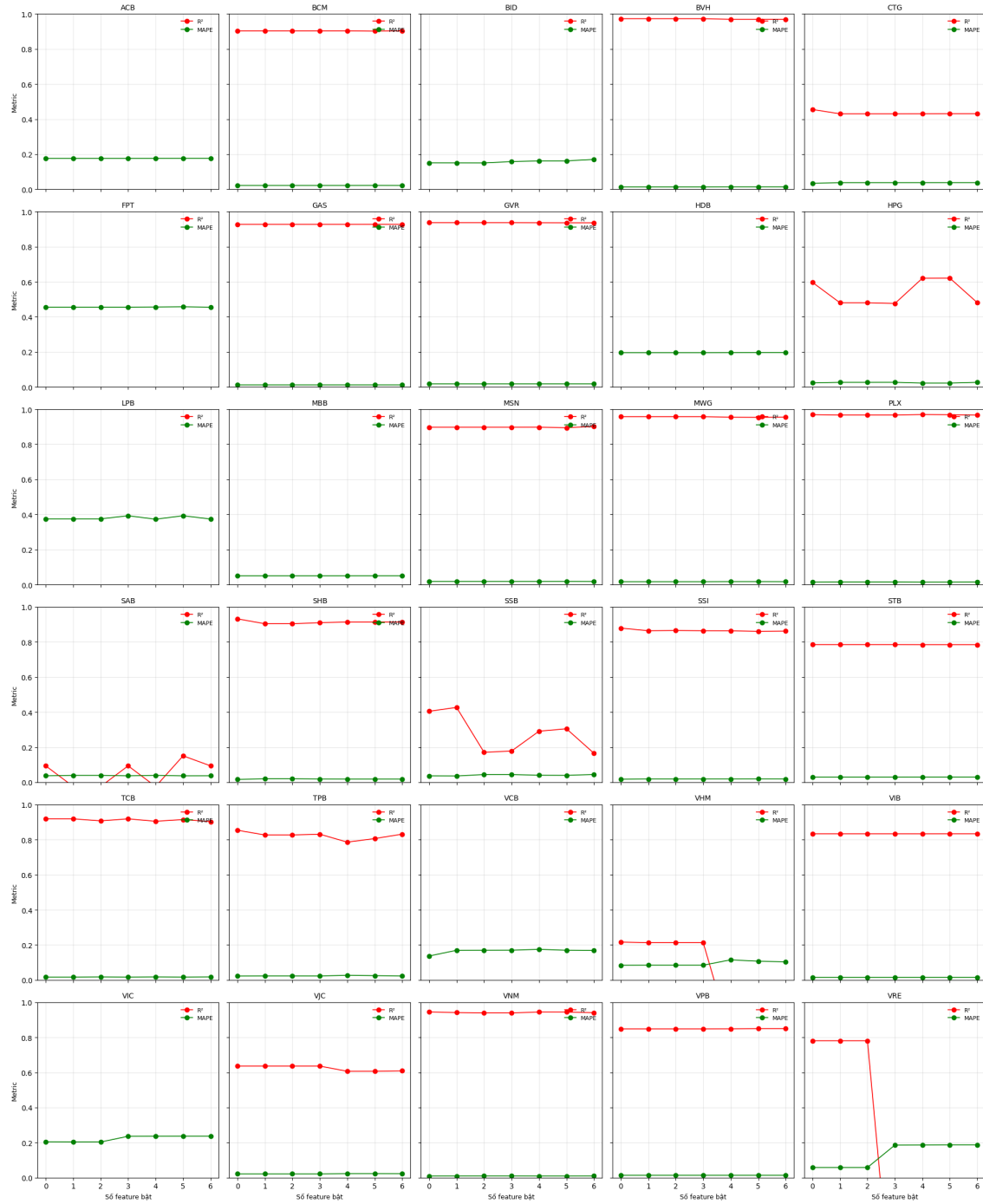
Với mô hình DT, cũng thực hiện tương tự, trình bày tại [Hình 7](#). Do giới hạn các siêu tham số như độ sâu cây mà ảnh hưởng của sự thay đổi đặc trưng không sâu sắc lắm.

Đối với ElasticNet, tham khảo [Hình 8](#), do quản lý đặc trưng hiệu quả nhờ cơ chế hiệu chỉnh nên không những mô hình không kém đi mà có thể tốt lên. Chứng minh rằng việc tăng tiến số lượng đặc trưng cần đi kèm với cơ chế hiệu chỉnh phù hợp.

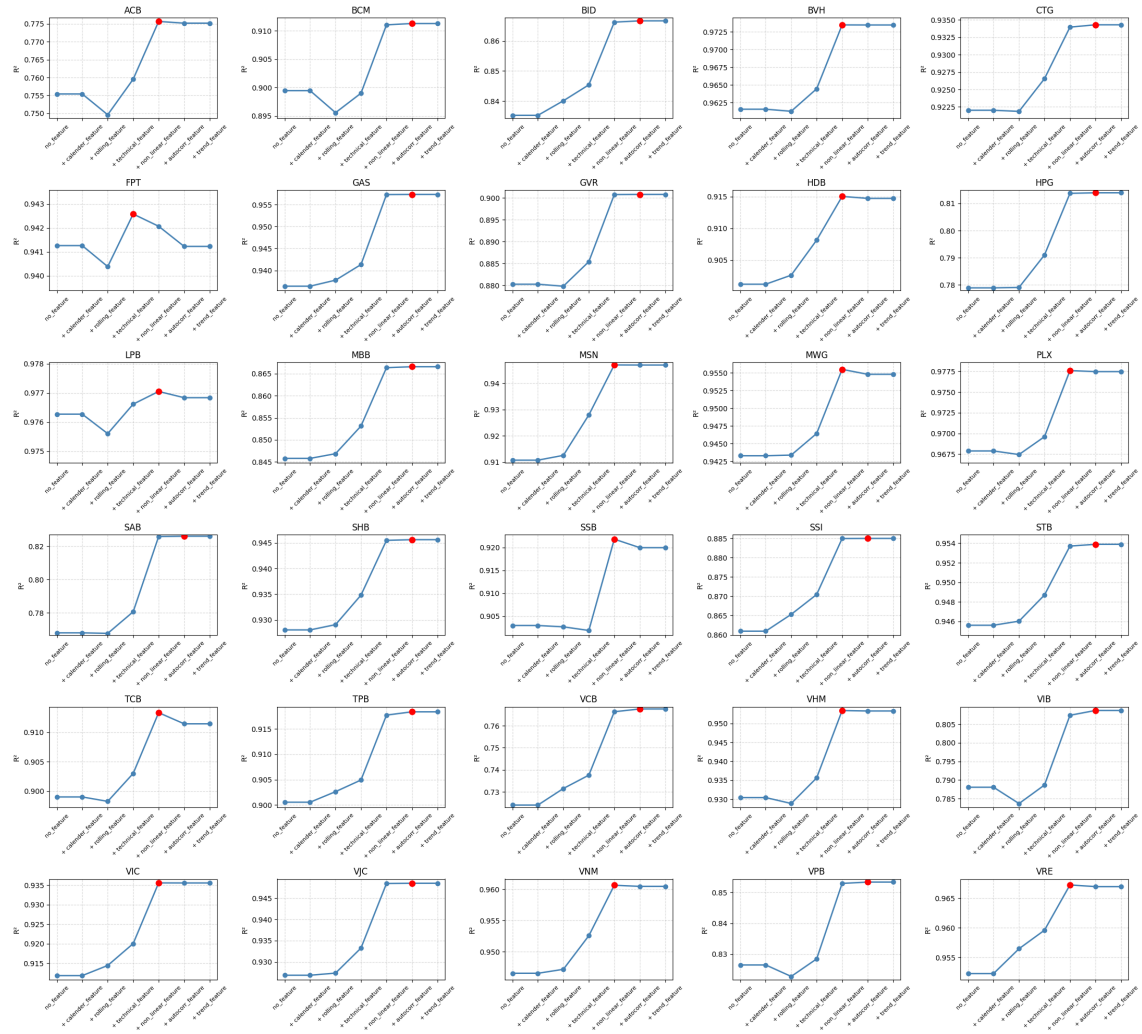


Hình 6: Ảnh hưởng của số lượng đặc trưng trong mô hình OLS

Ảnh hưởng số feature bật lên R^2 & MAPE (lag=30)



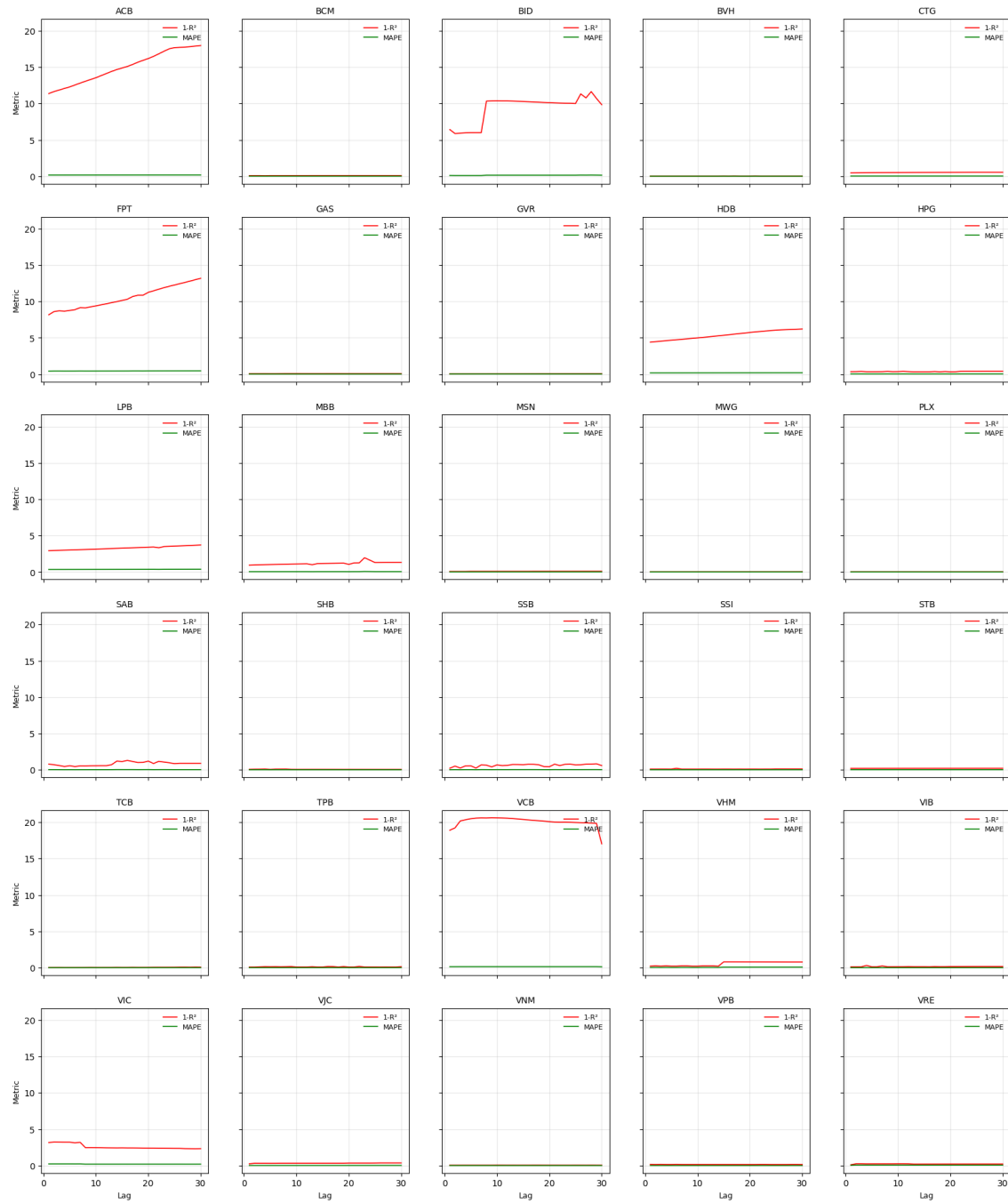
Hình 7: Ảnh hưởng của số lượng đặc trưng trong mô hình Decision Tree



Hình 8: Ảnh hưởng của số lượng đặc trưng trong mô hình ElasticNet

Thực nghiệm 2. Ảnh hưởng của trễ lên hiệu quả mô hình

Trong Hình 9, khi tăng tiến số lượng quan sát (tăng p), chúng tôi xem xét hiệu quả mô hình thay đổi thế nào. Kết quả có thể xem đi trên một số tập dữ liệu.



Hình 9: Ảnh hưởng của số ngày quan sát trong mô hình Decision Tree

Thực nghiệm 3. So sánh các mô hình tuyến tính

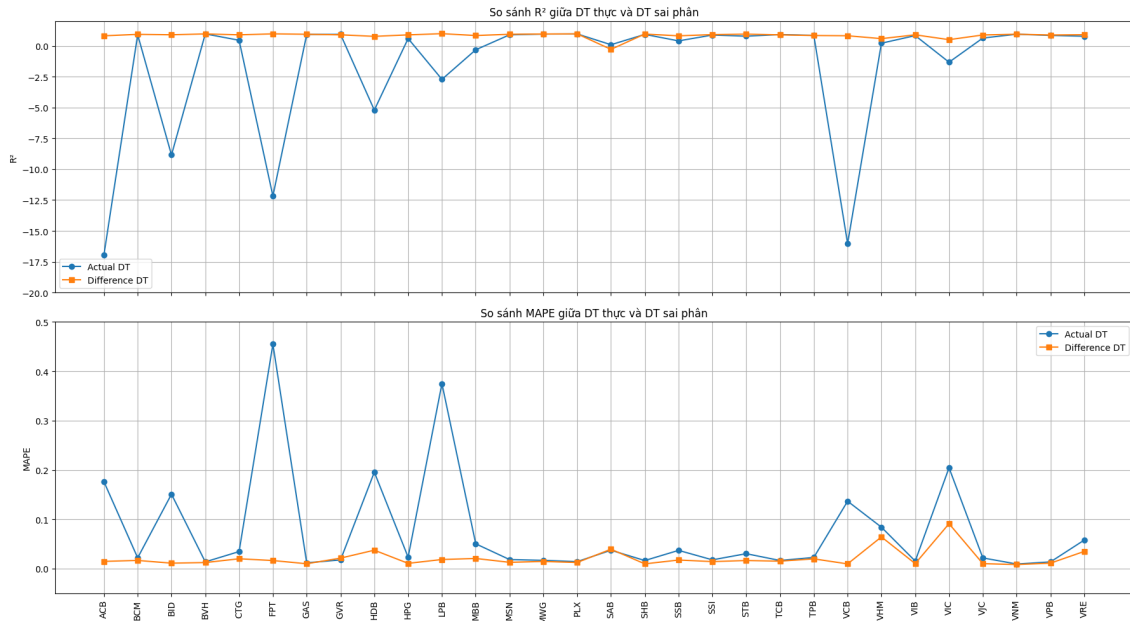
Kết quả trình bày tại [Bảng 1](#), cho thấy mô hình DT tỏ ra kém nhất, như nguyên nhân đã đề cập: mô hình cây không thể ngoại suy hiệu quả.

Bảng 1: Kết quả dự báo của nhóm mô hình tuyến tính

Mã	LR						DT	RVFL		
	OLS	Ridge	Lasso	Lasso (MT)	EN	EN (MT)		RVFL	dRVFL	edRVFL
ACB	0.94	0.94	0.93	0.93	0.76	0.83	-16.98	0.94	0.94	0.94
BCM	0.97	0.96	0.96	0.96	0.9	0.9	0.90	0.97	0.96	0.97
BID	0.93	0.94	0.94	0.94	0.84	0.87	-8.85	0.94	0.94	0.94
BVH	0.98	0.98	0.98	0.98	0.96	0.96	0.97	0.98	0.98	0.98
CTG	0.97	0.98	0.98	0.97	0.92	0.94	0.46	0.97	0.97	0.97
FPT	0.99	0.99	0.99	0.99	0.94	0.96	-12.18	0.99	0.99	0.99
GAS	0.97	0.97	0.97	0.96	0.94	0.94	0.93	0.97	0.97	0.97
GVR	0.95	0.96	0.96	0.96	0.88	0.90	0.94	0.95	0.95	0.96
HDB	0.97	0.98	0.98	0.98	0.90	0.93	-5.21	0.97	0.97	0.97
HPG	0.92	0.92	0.92	0.92	0.78	0.80	0.60	0.92	0.92	0.92
LPB	1.00	1.00	1.00	1.00	0.98	0.98	-2.71	1.00	1.00	1.00
MBB	0.96	0.96	0.95	0.96	0.85	0.88	-0.32	0.96	0.96	0.96
MSN	0.96	0.96	0.97	0.96	0.91	0.91	0.90	0.97	0.97	0.97
MWCG	0.98	0.98	0.98	0.98	0.94	0.95	0.96	0.98	0.98	0.98
PLX	0.98	0.98	0.98	0.98	0.97	0.97	0.97	0.98	0.98	0.98
SAB	0.95	0.95	0.95	0.94	0.77	0.85	0.09	0.95	0.95	0.95
SHB	0.97	0.97	0.97	0.97	0.93	0.93	0.93	0.97	0.97	0.97
SSB	0.95	0.95	0.96	0.95	0.90	0.92	0.41	0.95	0.95	0.95
SSI	0.95	0.95	0.95	0.95	0.86	0.88	0.88	0.95	0.95	0.95
STB	0.98	0.98	0.98	0.98	0.95	0.96	0.78	0.98	0.98	0.98
TCB	0.96	0.97	0.96	0.97	0.90	0.92	0.92	0.97	0.97	0.97
TPB	0.96	0.97	0.96	0.97	0.90	0.91	0.85	0.97	0.97	0.97
VCB	0.91	0.91	0.90	0.90	0.72	0.79	-16.03	0.91	0.91	0.91
VHM	0.98	0.98	0.98	0.98	0.93	0.95	0.22	0.98	0.98	0.98
VIB	0.92	0.93	0.92	0.93	0.79	0.83	0.83	0.93	0.93	0.93
VIC	0.98	0.98	0.98	0.98	0.91	0.94	-1.32	0.98	0.98	0.98
VJC	0.97	0.97	0.97	0.97	0.93	0.94	0.64	0.97	0.97	0.97
VNM	0.97	0.97	0.97	0.97	0.95	0.95	0.94	0.97	0.97	0.97
VPB	0.93	0.93	0.93	0.93	0.83	0.84	0.85	0.93	0.93	0.93
VRE	0.98	0.98	0.99	0.98	0.95	0.97	0.78	0.98	0.98	0.98
Rate	30/30	30/30	30/30	30/30	20/30	21/30	10/30	30/30	30/30	30/30
Avg	0.96	0.96	0.96	0.96	0.93	0.94	0.94	0.96	0.96	0.96

Thực nghiệm 4. Mô hình cây cho dự đoán sai phân bậc 1

Chúng tôi cải tiến mô hình cây, thay vì dự đoán chính xác giá trị thực, mô hình dự đoán sai phân bậc 1 của chuỗi, vì tính dừng và ổn định thống kê, nên mô hình cây trong trường hợp này không phải thực hiện nhiệm vụ ngoại suy phức tạp. Kết quả trình bày tại [Hình 10](#) chứng tỏ hiệu quả cao, giúp mô hình đạt kết quả ấn tượng.



Hình 10: So sánh 2 độ đo R-squared và MAPE giữa Decision Tree thông thường và sai phân

5.3 Mô hình học sâu

Thực nghiệm 5. So sánh các mô hình học sâu

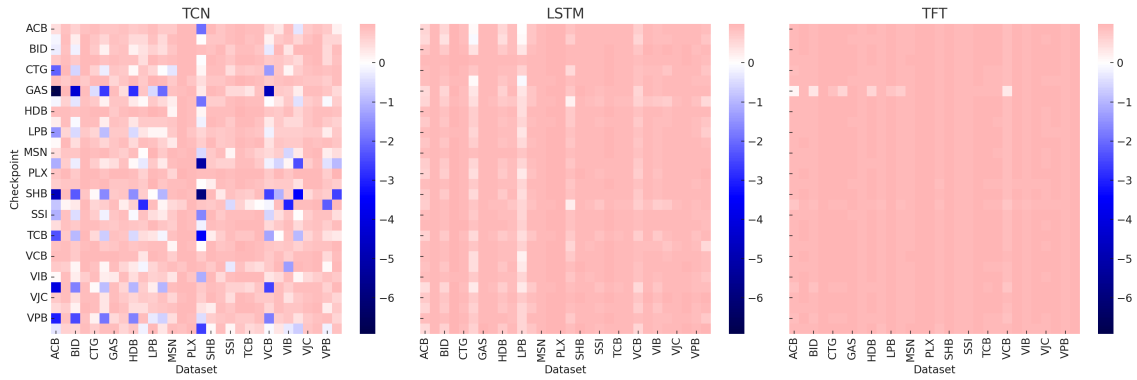
Các mô hình học sâu đề xuất có kết quả thể hiện tại [Bảng 2](#). Trong đó, TFT cho kết quả tốt nhất, tất nhiên đến từ việc chúng ta đã học thặng dư, nhằm dự đoán sai phân bậc 1. Ngoài ra, chúng tôi cố gắng cải thiện TCN và LSTM trên nền tảng của ensemble learning, kết hợp đầu ra của TCN và LSTM và coi chúng như những đặc trưng bổ sung vào nhóm đặc trưng đã có. Kết quả rõ ràng vượt trội hơn 2 mô hình nền tảng yếu và sánh ngang với TFT.

Bảng 2: Kết quả dự báo của các mô hình mạng

Mã	TCN		LSTM		TFT		Ensemble	
	R-squared	MAPE (%)	R-squared	MAPE (%)	R-squared	MAPE (%)	R-squared	MAPE (%)
ACB	0.45	2.84	0.56	2.61	0.92	0.79	0.91	0.88
BCM	0.86	2.73	0.47	2.87	0.97	1.12	0.97	1.08
BID	-0.23	5.43	0.78	1.68	0.93	0.87	0.88	1.36
BVH	0.92	2.06	0.92	0.76	0.98	0.91	0.98	0.90
CTG	0.62	4.47	0.85	1.32	0.97	1.03	0.93	1.61
FPT	0.96	2.00	0.88	1.03	0.97	1.76	0.99	1.09
GAS	0.90	1.33	0.47	2.85	0.97	0.67	0.96	0.73
GVR	0.77	3.44	0.57	2.58	0.96	1.38	0.95	1.40
HDB	0.92	1.81	0.86	1.26	0.97	0.94	0.97	0.96
HPG	0.71	1.95	0.74	1.90	0.92	0.88	0.92	0.88
LPB	0.10	19.04	0.88	1.06	0.99	1.67	0.99	1.36
MBB	0.60	3.49	0.79	1.64	0.95	0.93	0.94	1.09
MSN	0.91	1.63	0.86	1.27	0.96	0.96	0.97	0.95
MWG	0.91	2.33	0.67	2.21	0.98	1.03	0.97	1.27
PLX	0.96	1.60	0.61	2.44	0.98	0.93	0.98	1.01
SAB	0.02	4.97	0.85	1.29	0.88	1.48	0.94	0.96
SHB	0.93	1.53	0.69	2.13	0.97	0.77	0.97	0.79
SSB	0.83	1.81	0.67	2.27	0.96	0.96	0.95	0.88
SSI	0.86	1.91	0.89	0.99	0.95	0.98	0.95	0.88
STB	0.95	1.96	0.91	0.85	0.98	0.95	0.98	1.02
TCB	0.80	2.58	0.60	2.48	0.97	0.89	0.96	1.04
TPB	0.90	1.73	0.87	1.14	0.97	0.91	0.96	0.94
VCB	0.73	1.30	0.89	0.99	0.90	0.62	0.90	0.65
VHM	0.92	1.94	0.83	1.43	0.97	1.18	0.98	1.08
VIB	0.85	1.39	0.84	1.32	0.93	0.77	0.93	0.83
VIC	0.84	2.68	0.42	3.06	0.98	1.08	0.98	0.97
VJC	0.83	2.04	0.73	1.99	0.97	0.63	0.96	0.78
VNM	0.96	0.80	0.58	2.59	0.97	0.62	0.97	0.63
VPB	0.84	1.28	0.88	1.04	0.93	0.78	0.93	0.85
VRE	0.82	5.58	0.50	2.86	0.98	1.67	0.98	1.25
Avg	0.75	2.99	0.74	1.80	0.95	1.00	0.96	1.00

Thực nghiệm 6. Khả năng thích ứng của mô hình đã huấn luyện trên tập dữ liệu khác

Cuối cùng, chúng tôi kiểm thử tính “tổng quát hóa” của mô hình, như trong 11. Điều này có nghĩa là, chúng tôi mang mô hình đã huấn luyện trên mã chứng khoán X, sau đó kiểm thử trên dữ liệu của mã chứng khoán Y. Mô hình có kết quả ấn tượng nhất thuộc về TFT, khi hầu hết các mô hình tuân theo kiến trúc này đều chuyển giao tốt sang một tập dữ liệu khác.



Hình 11: Bản đồ nhiệt khi thay đổi mô hình đã huấn luyện trên tập dữ liệu khác

6 Kết luận

Tổng kết lại, bài toán dự báo chuỗi thời gian được phát biểu một cách tổng quát là quá trình xây dựng mô hình học máy nhằm dự đoán các giá trị trong tương lai dựa trên các quan sát lịch sử của một chuỗi số liệu. Trong quá trình nghiên cứu, chúng tôi đã xây dựng một hệ thống mô hình phong phú, bao gồm cả các phương pháp truyền thống và hiện đại, đồng thời thực hiện đánh giá và so sánh toàn diện giữa các mô hình này theo nhiều tiêu chí như độ chính xác, độ ổn định, và khả năng tổng quát hóa. Các kết quả thực nghiệm cho thấy mô hình đề xuất không chỉ đạt hiệu quả cao mà còn có tiềm năng ứng dụng thực tiễn mạnh mẽ, đồng thời có thể đóng vai trò như những mô hình cơ sở đáng tin cậy cho các nghiên cứu và triển khai nâng cao trong tương lai.